



UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
DISCIPLINA DE SISTEMAS OPERACIONAIS

PROCESSOS, THREADS, ESCALONAMENTO E
INFERÊNCIA LOCAL COM OLLAMA
Equipe Kwen

SÃO CRISTÓVÃO

2026

SUMÁRIO

1. Desenvolvimento dos testes e análise	1
1.1 Qual trilha de aplicação foi selecionada e por que ela é apropriada para a disciplina?	1
1.2 Qual modelo foi selecionado, quais são seus parâmetros e por que ele foi escolhido?	1
1.3 Como formato, quantização, contexto e tamanho do modelo afetaram os requisitos de armazenamento, RAM ou VRAM?	2
1.4 Quais processos e threads foram observados durante a inicialização e a inferência?	6
1.5 Qual é a relação de parentesco e responsabilidade entre os processos e threads envolvidos?	12
1.6 Quais chamadas de sistema foram relevantes para a carga do modelo, para a leitura de dados, para a comunicação ou para a geração de logs?	17
1.7 O aumento da concorrência melhorou a responsividade ou a vazão?	27
1.8 Houve sinais de competição por CPU, memória, armazenamento, GPU, rede?	33
1.9 Como entradas ou contextos maiores afetaram o desempenho e o uso de recursos?	40
1.10 Como a arquitetura local contribui para privacidade, governança e controle de dados?	43
1.11 Como o sistema poderia se comportar diante de múltiplos usuários concorrentes?	46
1.12 Quais limitações do hardware, do runtime, da camada de aplicação e do método de avaliação foram identificadas?	48
1.13 Quais informações geradas por IA precisaram ser verificadas, corrigidas ou descartadas?	53
2. Referências	57
3. Declaração de Uso de Inteligência Artificial	58

1. Desenvolvimento dos testes e análise

1.1 Qual trilha de aplicação foi selecionada e por que ela é apropriada para a disciplina?

A trilha de aplicação utilizada foi a A do documento passado pelo professor, que utiliza o Ollama em conjunto do Open WebUi. Além disso, o Open WebUi foi executado em container, tendo sido escolhido o modelo Qwen2.5-1.5B-Instruct.

Entendemos ser apropriada para disciplina pois nos consente observar e analisar diretamente múltiplos conceitos, de processamento da área de forma integrada.

Ele permite ao aluno, como descrito no item 6 do trabalho: ver o gerenciamento de processamento e threads, analisar a comunicação HTTP via API local, portas, sockets e conexões estabelecidas entre a interface e o runtime, investigar a execução nativa versus em contêiner (Docker), avaliando como o sistema operacional lida com o isolamento de recursos, análise de volumes mapeados, diretórios persistentes e logs gerados pelas aplicações em disco, além de cenário prático para monitoramento e consumo real e os picos de memória durante a execução do Open WebUi no sistema apontado. Conforme veremos nas análises.

1.2 Qual modelo foi selecionado, quais são seus parâmetros e por que ele foi escolhido?

O modelo foi o Qwen2.5-1.5B-Instruct, e seus parâmetros são de 1.5 bilhões (1.5B) de acordo com sua documentação [1], sendo escolhido pelo motivo de está dentro dos máximo de 10 bilhões de parâmetros estabelecido no trabalho, além do que, é leve, pois segundo sua documentação é uma versão quantizada em Q4_K_M¹, que roda no [Ollama](#) consumindo apenas cerca de ~935 MB a 986 MB (RAM ou VRAM), pois o ecossistema llama.cpp carrega esses pesos diretamente na memória, o consumo base do modelo inicia nessa faixa de ~0,9 GB. o que o torna ideal para testes de concorrência, mesmo em computadores sem placa de vídeo dedicada. [2, 3, 4]. Veja figura 1 abaixo:

¹ Quantização Q4 (4 bit) é uma técnica de compressão da estrutura usada por linguagens (LLMs) que reduz a precisão dos números representantes dos parâmetros da rede neural, ao invés de usar os padrões de 16 e 32 bits, os modelos são mapeados e armazenados em inteiros de apenas 4 bits. [2]

```
matheus@matheusnb:/mnt/c/WINDOWS/system32$ ollama run qwen2.5:1.5b-instruct-q4_K_M
pulling manifest
pulling 183715c43589: 100% ▢ 985 MB/986 MB 70 MB/s 0s
pulling 66b9ea09bd5b: 100% ▢ 68 B
pulling eb4402837c78: 100% ▢ 1.5 KB
pulling 832dd9e00a68: 100% ▢ 11 KB
verifying sha256 digest
writing manifest
success
```

Figura 1: Tamanho utilizado em MB de memória para os ambientes com GPU e sem GPU.

1.3 Como formato, quantização, contexto e tamanho do modelo afetaram os requisitos de armazenamento, RAM ou VRAM?

Os testes foram realizados em dois ambientes, com GPU que teve uma NVIDIA GeForce RTX 3050 com 6144 MiB de VRAM. Aonde os registros das três repetições (A-1, A-2 e A-3) foram armazenados individualmente, permitindo comparar o comportamento da GPU e o desempenho do modelo nas mesmas condições experimentais. O ambiente utilizado foi:

- Sistema Operacional: Linux ("Ubuntu 24.04.2 LTS")
Processador: AMD Ryzen 5 5600GT with Radeon Graphics (6 núcleos e 12 Threads)
Placa de vídeo: NVIDIA GeForce RTX 3050, 6144 MiB, 595.84
Armazenamento: 1 TB Total
Memória RAM (WSL): 15 GB Total
Runtime de IA: ollama version is 0.33.3

E sem GPU por incompatibilidade dos drivers da AMD com o wsl, com essa configuração de hardware:

- Sistema Operacional: Linux (Ubuntu via WSL2 no Windows) Kernel: 6.6.114.1-microsoft-standard-WSL2
Processador: AMD Ryzen 5 5500 (6 núcleos e 12 Threads)
Placa de vídeo: AMD RX 7600 8GB Armazenamento: 1007G Total
Memória RAM (WSL): 15 GB Total
Runtime de IA: Ollama (Execução em CPU apenas, devido a limitações do wsl referentes a GPU da amd)

Quanto ao armazenamento não houve alteração os dois ambientes consumiram 500gb.

No modelo com GPU observando as figuras 2 e 3 abaixo, para os testes A e B, mostram a coluna context de tokens permanecia com 4096 antes e depois da execução:

- Estado do Ollama antes:

```
===== A-1: OLLAMA PS ANTES =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	About a minute from now

- Estado do Ollama depois:

```
===== A-1: OLLAMA PS DEPOIS =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	4 minutes from now

Figura 2: estado antes e depois da execução no teste A.

- Estado do Ollama antes/depois: `ollama-ps-antes.txt`,

```
===== A-1: OLLAMA PS ANTES =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	About a minute from now

`ollama-ps-depois.txt`

```
===== B-1: OLLAMA PS DEPOIS =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	4 minutes from now

Figura 3: estado antes e depois da execução no teste B.

Já para o teste C o context de tokens foi alterado para 8192, veja figura 4 seguir.

- Estado do Ollama antes/depois: `ollama-ps-antes.txt`

```
===== C-1: OLLAMA PS ANTES =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	About a minute from now

, `ollama-ps-depois.txt`

```
===== C-1: OLLAMA PS DEPOIS =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.4 GB	100% GPU	8192	4 minutes from now

Figura 4: estado antes e depois da execução no teste C.

Percebemos que com 4096 de tokens (configuração A e B) o modelo ocupou 1,2GB de VRAM já com 8192 de tokens (configuração C) o modelo ocupou 1,4GB de VRAM dada à coluna SIZE, isso por conta do contexto expandido do

teste C. O tempo de carregamento também 3,319821 s dado a alocação de um contexto maior, pois no A foi de 0,000618 s e do B 1,965338 s.

Já para o modelo sem GPU o uso foi exclusivo da RAM dada a limitações de drivers e incompatibilidade da GPU AMD com o WSL2, não possível usar a VRAM da placa de vídeo AMD RX 7600 8GB sendo identificado pelo sistema operacional apenas o Microsoft Corporation Basic Render Driver. A execução do free -h mostrava 4.0 GiB em uso e 14 GiB disponíveis/cache. Segue figura 5, demonstrando:

Memória RAM

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/System32$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	15Gi	611Mi	14Gi	3.5Mi	474Mi	14Gi
Swap:	4.0Gi	0B	4.0Gi			

Figura 5: Consumo de memória RAM.

Veja que no teste das configurações A, B e C foi utilizado 100% de CPU e um contexto de 4096 de tokens, isso não se modificou, vejam as figuras 6, 7 e 8 abaixo.

Antes do Ollama

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/System32$ ollama ps
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b	65ec06548149	1.2 GB	100% CPU	4096	4 minutes from now

Depois do Ollama

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/System32$ ollama ps
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b	65ec06548149	1.2 GB	100% CPU	4096	4 minutes from now

Figura 6: estado antes e depois da execução no teste A.

Antes do Ollama

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/System32$ ollama ps
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b	65ec06548149	1.2 GB	100% CPU	4096	4 minutes from now

Depois do Ollama

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/System32$ ollama ps
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b	65ec06548149	1.2 GB	100% CPU	4096	4 minutes from now

Figura 7: estado antes e depois da execução no teste B.

Antes do Ollama

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/System32$ ollama ps
NAME          ID          SIZE    PROCESSOR  CONTEXT  UNTIL
qwen2.5:1.5b  65ec06548149  1.2 GB   100% CPU   4096     4 minutes from now
```

Depois do Ollama

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/System32$ ollama ps
NAME          ID          SIZE    PROCESSOR  CONTEXT  UNTIL
qwen2.5:1.5b  65ec06548149  1.2 GB   100% CPU   4096     4 minutes from now
```

Figura 8: estado antes e depois da execução no teste C.

O impacto na RAM vs. VRAM mais crítico documentado foi a incompatibilidade dos drivers da GPU AMD com o WSL2, conforme colocamos no parágrafo acima. Por conta dessa limitação, a placa de vídeo AMD RX 7600 de 8GB não pôde utilizar sua VRAM para processamento, forçando o Ollama a rodar com 100% de uso de CPU. Consequentemente, todo o carregamento do modelo (1.2 GB) e o contexto das requisições foram jogados inteiramente para a memória RAM do sistema (que possuía ~15 GB utilizáveis no WSL). O relatório destaca explicitamente que na C, mesmo existindo VRAM disponível, o sistema utilizou estritamente a CPU devido a essa limitação técnica.

Por fim, vemos que quando utilizado a VRAM o processamento fica melhor pois utiliza mais recursos, vejam nos gráficos abaixo que VRAM x RAM que ele consegue utilizar mais memória de VRAM deixando o processamento mais rápido, principalmente no teste C que exige mais, bem como, consegue abrir mais tokens, em quanto que no ambiente sem GPU que só utilizou a RAM isso se mantém estável para todos os cenários.

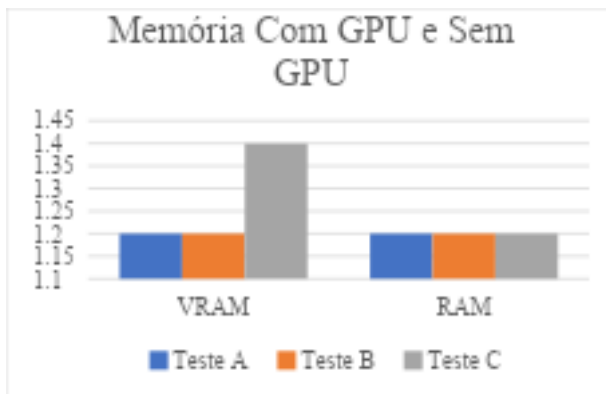


Gráfico 1: Memória.

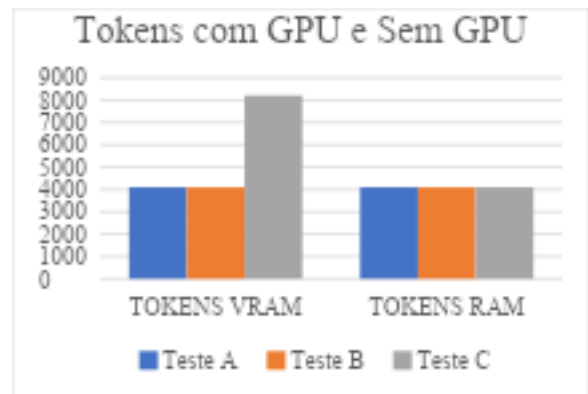


Gráfico 2: Tokens.

1.4 Quais processos e threads foram observados durante a inicialização e a inferência?

Para os processos de inferência e inteligência artificial o Ollama utilizou o PID 78507 que é o processo de backend do Ollama efetivamente carregado. No modelo com GPU vejam que ele é executa a inferência, que tem o tipo C (compute). Nos três casos ele se manteve com 1208 MiB, conforme figura XX e XX.

Para os processos Processos do Sistema Operacional e Background (Interface Gráfica), estes processos já estavam ativos antes da execução (tipo G - Graphics) e continuaram rodando em paralelo, dividindo o ecossistema da GPU com o Ollama, Interface e Janelas: Xorg (PID 2761) e gnome-shell (PID 2965), gerenciador de arquivos: nautilus (PID 76850) e aplicações de terceiros: processos relacionados ao steam e steamwebhelper (PIDs 6864, 7086, 7117). Outros subprocessos de computação: instâncias adicionais em background identificadas por hashes de UUID (PIDs 5257 e 79334). Como também figura vemos nas figuras 9 e 10 abaixo.

Estado da GPU antes:

```
===== A-1: NVIDIA ANTES =====
Thu Sep 10 12:09:17 2026
```

NVIDIA-SMI 595.84			Driver Version: 595.84			CUDA Version: 13.2		
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp	Perf	Memory-Usage	GPU-Util	Compute M.	MIG	M.	
		Pwr:Usage/Cap						
0	NVIDIA GeForce RTX 3050	Off	00000000:10:00.0	On			N/A	
0%	47C	P8	1982MiB / 6144MiB	1%	Default		N/A	

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	Usage
	ID	ID					
0	N/A	N/A	2761	G	/usr/lib/xorg/Xorg	279MiB	
0	N/A	N/A	2965	G	/usr/bin/gnome-shell	65MiB	
0	N/A	N/A	5257	G	...rack-uuid=3190708988185955192	147MiB	
0	N/A	N/A	6125	G	...0909-090050.001000-production	30MiB	
0	N/A	N/A	6864	G	...share/Steam/ubuntu12_32/steam	3MiB	
0	N/A	N/A	7086	G	./steamwebhelper	2MiB	
0	N/A	N/A	7117	G	...am/ubuntu12_64/steamwebhelper	84MiB	
0	N/A	N/A	76850	G	/usr/bin/nautilus	13MiB	
0	N/A	N/A	78507	C	...local/lib/ollama/llama-server	1208MiB	
0	N/A	N/A	79334	G	...rack-uuid=3190708988185955192	42MiB	

Figura 9: processos durante a inicialização e inferência, nos 3 tipos de teste.

Estado da GPU depois:

```

===== A-1: NVIDIA DEPOIS =====
Thu Sep 10 12:09:22 2026

```

NVIDIA-SMI 595.84			Driver Version: 595.84			CUDA Version: 13.2		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
0	NVIDIA GeForce RTX 3050		Off	00000000:10:00.0	On		N/A	
0%	56C	P2	59W / 70W	1965MiB / 6144MiB		97%	Default	N/A

Processes:							
GPU	GI ID	CI ID	PID	Type	Process name	GPU Memory Usage	
0	N/A	N/A	2761	G	/usr/lib/xorg/Xorg	279MiB	
0	N/A	N/A	2965	G	/usr/bin/gnome-shell	65MiB	
0	N/A	N/A	5257	G	...rack-uuid=3190708988185955192	130MiB	
0	N/A	N/A	6125	G	...0909-090050.001000-production	30MiB	
0	N/A	N/A	6864	G	...share/Steam/ubuntu12_32/steam	3MiB	
0	N/A	N/A	7086	G	./steamwebhelper	2MiB	
0	N/A	N/A	7117	G	...am/ubuntu12_64/steamwebhelper	84MiB	
0	N/A	N/A	76850	G	/usr/bin/nautilus	13MiB	
0	N/A	N/A	78507	C	...local/lib/ollama/llama-server	1208MiB	
0	N/A	N/A	79334	G	...rack-uuid=3190708988185955192	42MiB	

Figura 10: processos durante a inicialização e inferência, nos 3 tipos de teste.

A variação da GPU em % se deu assim:

- Estado de Repouso (Idle): Antes de iniciar a inferência, a utilização da GPU oscila entre 1% e 5%. O processo llama-server fica em background apenas retendo o modelo na VRAM, sem demandar processamento ativo.
- Fase de Inicialização: Nos primeiros milissegundos após o envio do prompt, a utilização sobe levemente (na faixa de 6%) enquanto os buffers e o contexto inicial são preparados.
- Fase de Inferência (Geração de Texto): Assim que a geração de tokens começa, a utilização salta bruscamente para patamares estabilizados entre 93% e 97%.
- Retorno ao Repouso: Assim que o último token é entregue (sucesso do Ollama), a utilização cai imediatamente de volta para 1%, reduzindo também o consumo elétrico e a dissipação térmica da GPU.

Para comprovar vejam os logs na figura 11 abaixo.

```
===== B-1: GPU =====
timestamp, memory.used [MiB], utilization.gpu [%], utilization.memory [%], power.draw [W], temperature.gpu
2026/09/10 20:28:48.927, 673 MiB, 5 %, 13 %, 16.08 W, 42
2026/09/10 20:28:49.135, 672 MiB, 5 %, 13 %, 16.08 W, 42
2026/09/10 20:28:49.335, 672 MiB, 6 %, 12 %, 16.08 W, 42
2026/09/10 20:28:49.535, 672 MiB, 6 %, 12 %, 16.08 W, 42
2026/09/10 20:28:49.758, 672 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:49.959, 755 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:50.159, 689 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:50.359, 672 MiB, 4 %, 2 %, 16.26 W, 43
2026/09/10 20:28:50.560, 705 MiB, 4 %, 2 %, 16.26 W, 43
2026/09/10 20:28:50.760, 763 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:50.960, 1699 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:51.160, 1699 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:51.360, 1893 MiB, 64 %, 5 %, 21.75 W, 43
2026/09/10 20:28:51.560, 1893 MiB, 64 %, 5 %, 21.75 W, 43
2026/09/10 20:28:51.760, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:51.961, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:52.161, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:52.361, 1893 MiB, 96 %, 94 %, 49.53 W, 49
2026/09/10 20:28:52.561, 1892 MiB, 96 %, 94 %, 49.53 W, 49
2026/09/10 20:28:52.761, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:52.961, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:53.161, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:53.362, 1892 MiB, 95 %, 93 %, 68.66 W, 52
2026/09/10 20:28:53.562, 1892 MiB, 95 %, 93 %, 68.66 W, 52
2026/09/10 20:28:53.762, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:53.962, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:54.162, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:54.362, 1892 MiB, 97 %, 96 %, 68.67 W, 53
2026/09/10 20:28:54.562, 1892 MiB, 97 %, 96 %, 68.67 W, 53
2026/09/10 20:28:54.763, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:54.963, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:55.163, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:55.363, 1892 MiB, 97 %, 95 %, 68.72 W, 54
2026/09/10 20:28:55.563, 1892 MiB, 97 %, 95 %, 68.72 W, 54
2026/09/10 20:28:55.763, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:55.963, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:56.164, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:56.364, 1892 MiB, 97 %, 95 %, 68.47 W, 55
2026/09/10 20:28:56.564, 1892 MiB, 97 %, 95 %, 68.47 W, 55
2026/09/10 20:28:56.764, 1892 MiB, 97 %, 95 %, 68.93 W, 55
2026/09/10 20:28:56.964, 1892 MiB, 97 %, 95 %, 68.93 W, 55
```

Figura 11: com o log de evolução da execução.

Já o impacto na Memória RAM geral do Sistema, embora o modelo de IA seja alocado inteiramente na memória de vídeo (VRAM), o processo do sistema hospedeiro precisa gerenciar a execução da aplicação e buffers de entrada/saída, o que consome memória RAM convencional.

A análise das métricas coletadas nas configurações A, B e C, ilustra o impacto da RAM antes da inferência: O sistema registrava 8,5 GiB ocupados de um total de 15 GiB. E a RAM depois da inferência: A ocupação subiu para 9,2 GiB. Impacto Real: Houve um acréscimo líquido de 0,7 GiB (700 MiB) na memória RAM global do sistema. Esse aumento representa o espaço em memória exigido pelo ecossistema do Ollama e pelas bibliotecas de runtime do Linux para sustentar as requisições em paralelo e o tráfego dos tokens gerados de volta para o cliente.

Estado da RAM antes/depois: `memoria-antes.txt`

```

===== B-1: MEMÓRIA ANTES =====
              total      used      free      shared  buff/cache  available
Mem:          15Gi      8.5Gi      1.2Gi      396Mi      6.5Gi      7.0Gi
Swap:          4.0Gi      1.4Gi      2.6Gi

```

, `memoria-depois.txt`

```

===== B-1: MEMÓRIA DEPOIS =====
              total      used      free      shared  buff/cache  available
Mem:          15Gi      9.2Gi      557Mi      602Mi      6.7Gi      6.3Gi
Swap:          4.0Gi      1.4Gi      2.6Gi

```

Figura 12: Impacto na RAM para os testes A e B.

Estado da RAM antes/depois: `memoria-antes.txt`

```

===== C-1: MEMÓRIA ANTES =====
              total      used      free      shared  buff/cache  available
Mem:          15Gi      9.1Gi      610Mi      609Mi      6.7Gi      6.4Gi
Swap:          4.0Gi      1.4Gi      2.6Gi

```

, `memoria-depois.txt`

```

===== C-1: MEMÓRIA DEPOIS =====
              total      used      free      shared  buff/cache  available
Mem:          15Gi      9.1Gi      648Mi      627Mi      6.7Gi      6.4Gi
Swap:          4.0Gi      1.4Gi      2.6Gi

```

Figura 13: Impacto na RAM para o teste C.

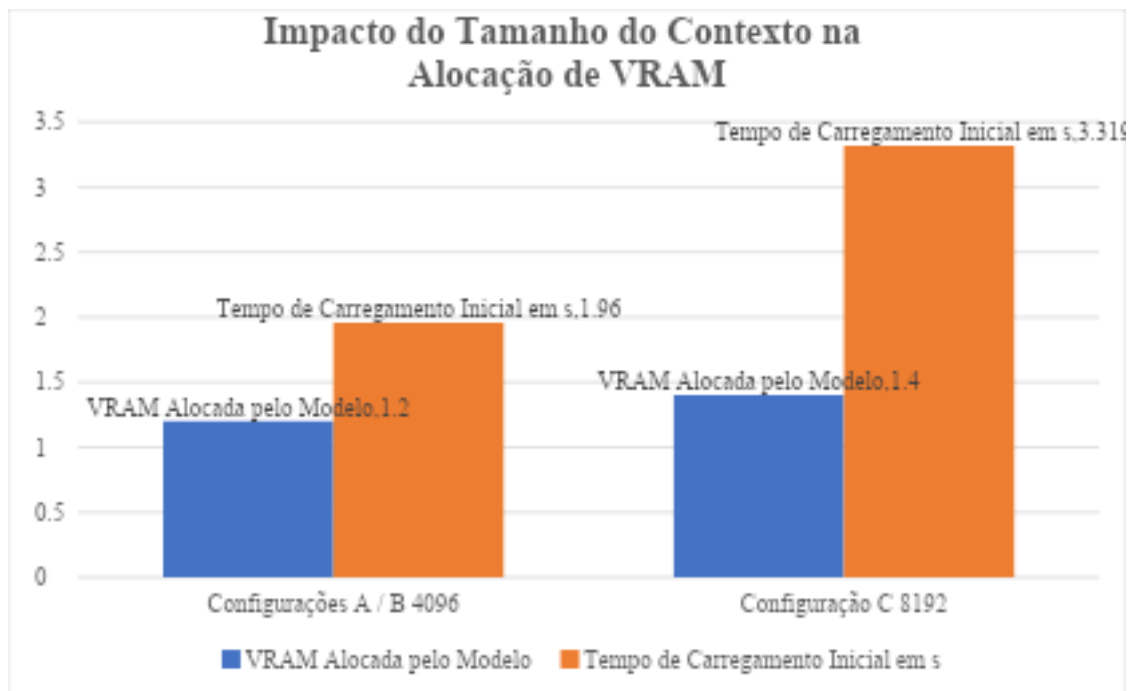


Gráfico 3: Impacto do Tamanho do Contexto na Alocação de VRAM

Olhando o gráfico 3 acima vemos:

- Crescimento Linear Prévio: Ao expandir o contexto de 4096 para 8192 tokens (um aumento de 100%), o consumo de VRAM saltou de 1,2 GB para 1,4 GB.
- Impacto do KV Cache: Esse acréscimo de 200 MB é provocado pela alocação estática das tabelas de chaves e valores (Key-Value Cache ou KV Cache) que o Ollama cria previamente na VRAM para dar suporte a sequências textuais mais longas.
- Penalidade de Inicialização: Embora a velocidade de geração em si permaneça alta (122 tokens/s), a alocação desse espaço extra de 8192 tokens gerou um custo temporal maior durante a inicialização (subindo para 3,31 segundos).

Já para o ambiente sem GPU, os processos e threads observados se comportaram da seguinte forma:

- Processo Principal (PID 1294): Durante a inicialização e execução, foi identificado um processo pai unificado do Ollama sob o comando `/usr/local/bin/ollama serve`.
- Threads de Execução: O comando `ps -ef | grep ollama` listou pelo menos 15 threads simultâneas atreladas ao mesmo processo principal (PID 1294). Cada uma dessas ramificações de tarefas individuais (como as de PID secundário 1301, 1302, 1303, até 1365) gerencia uma parte da concorrência do serviço, permitindo atender requisições paralelas.
- Uso de Processadores: Como o hardware possui um processador AMD Ryzen 5 5500 com 6 núcleos e 12 threads de processamento físico/lógico, e a GPU não foi acionada por incompatibilidade no WSL, a inferência do modelo consumiu 100% da CPU, dividindo o processamento matemático do modelo entre as threads nativas do sistema.

Com base nos dados e logs fornecidos, segue um breve comparativo entre a execução do Ollama no ambiente Com GPU e no ambiente Sem GPU:

- **Eficiência e Isolamento (Com GPU):** O modelo roda de forma dedicada na memória de vídeo (VRAM). O uso de processamento da GPU salta bruscamente durante a geração de texto, mas retorna para quase zero (1%) assim que a inferência termina, minimizando o consumo elétrico geral.
- **Sobrecarga do Hospedeiro (Sem GPU):** Pela falta de compatibilidade ou aceleração no WSL, o sistema é forçado a usar 100% da capacidade da CPU. Embora use 15 threads simultâneas para tentar paralelizar o cálculo matemático, isso deixa o processador principal totalmente estrangulado durante a execução.

Abaixo temos uma tabela de comparativo entre os dois ambientes.

Critério	Ambiente Com GPU	Ambiente Sem GPU (CPU / WSL)
Componente Principal	NVIDIA GeForce RTX 3050	AMD Ryzen 5 5500 (6 núcleos / 12 threads)
Carga de Processamento	Concentrada na GPU, atingindo 93% a 97% na inferência.	Consome 100% da CPU, dividida entre as threads simultâneas.
Alocação do Modelo	Totalmente carregado na VRAM (1208 MiB).	Gerenciado via processo pai e múltiplas threads na RAM.
Paralelismo	Executado como tipo C (Compute) pelo llama-server.	Executado via ollama serve com pelo menos 15 threads.
Impacto no Sistema	Divide o ecossistema da GPU com processos gráficos (Xorg, GNOME).	Sobrecarrega os processadores principais do computador hospedeiro.

Tabela 1: Comparativo entre os ambientes com GPU e sem GPU.

1.5 Qual é a relação de parentesco e responsabilidade entre os processos identificados?

A divisão de responsabilidades, no Linux, através do Ollama Daemon (Processo Pai), gerenciamento de Alto Nível e Interface API e llama-server (Processo Filho) inferência, computação e execução na GPU.

Para o ambiente com GPU as evidências do papel de "Gerenciador" do Ollama (Pai), controla o ciclo de vida e a configuração do modelo, veja nos dados

abaixo é o ollama quem cronometra o "Tempo de carregamento", "Tempo total" e dita as regras do contexto.

Dados registrados pela API do Ollama no teste A:

- Tempo total Ollama: 3,536759 s
- Tempo de carregamento: 0,000618 s

Dados registrados pela API do Ollama no teste B:

- Tempo total Ollama: 12,718010 s
- Tempo de carregamento: 1,965563 s

Dados registrados pela API do Ollama no teste C:

- Tempo total Ollama: 7,029404 s
- Tempo de carregamento: 3,319821 s (Nota: Houve aumento no tempo de carga devido à alocação de um contexto maior)

Já ele no papel de "Executor" do llama-server (Filho), vemos que quem faz o trabalho pesado de reter e processar o modelo na GPU é o llama-server, enquanto o Ollama monitora de fora. Vemos isso observando que o consumo isolado do binário foi de “llama-server: 1208 MiB de VRAM”. Analisando as figuras 14 e 15, vemos que vai de 1,2 de size para os ambientes A e B e 1,4 para o ambiente C, deixando sempre o consumo com 100% de GPU. Essa diferença de visualização no terminal prova a hierarquia, ou seja, o usuário interage com o comando do Ollama (ollama ps), mas quem está de fato alocado no hardware (conforme o nvidia-smi) é o subprocesso filho llama-server.

Estado do Ollama antes:

===== A-1: OLLAMA PS ANTES =====					
NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	About a minute from now

Estado do Ollama depois:

===== A-1: OLLAMA PS DEPOIS =====					
NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	4 minutes from now

Figura 14: antes e depois da execução para os testes A e B.

Estado do Ollama antes/depois: `ollama-ps-antes.txt`

```
===== C-1: OLLAMA PS ANTES =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.2 GB	100% GPU	4096	About a minute from now

, `ollama-ps-depois.txt`

```
===== C-1: OLLAMA PS DEPOIS =====
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5:1.5b-instruct	65ec06548149	1.4 GB	100% GPU	8192	4 minutes from now

Figura 15: antes e depois da execução para o teste C.

Já para o ambiente sem GPU o processo Pai (PID 1294) é o processo principal do serviço, iniciado pelo comando `/usr/local/bin/ollama serve`. Ele possui o ID de Processo Pai (PPID) igual a 1, o que significa que ele foi adotado diretamente pelo sistema operacional (`init/systemd`) ao rodar em segundo plano como um daemon. Que tem a responsabilidade de gerenciar a infraestrutura do Ollama, escutar requisições de rede na porta padrão 11434, gerenciar o ciclo de vida do modelo `qwen2.5:1.5b` e coordenar a distribuição de tarefas. Veja na figura 16.

`ps -eLf | grep ollama`

```
biaah@DESKTOP-IDA0AM7:/mnt/c/Users/Biaah$ ps -eLf | grep ollama
ollama 1294 1 1294 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1301 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1302 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1303 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1304 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1305 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1308 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1309 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1312 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1313 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1314 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1315 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1317 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1363 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
ollama 1294 1 1365 0 15 11:31 ? 00:00:00 /usr/local/bin/ollama serve
biaah 15150 15117 15150 0 1 11:44 pts/5 00:00:00 grep --color=auto ollama
```

Figura 16: Comando usado para verificar os processos criados (PID 1294).

Os Processos Filhos / Threads de Execução na listagem, aparecem múltiplas linhas repetindo o PID 1294 associado a diferentes identificadores de threads (como as sequências 1301, 1302, 1303 até 1365). Que tem como responsabilidade, executar o processamento pesado e paralelo. Como o ambiente realizou testes com 3 requisições simultâneas e 3 conversas paralelas, essas threads dividem o trabalho concorrente de inferência na CPU. Como vimos também na XX acima.

Confirmamos isso com o resumo strace, abaixo, que nos mostra a chamada de sistema futex consumiu 88.29% do tempo total de execução (mais de 74 segundos). O futex (Fast Userspace Mutex) serve justamente para fazer a sincronização e comunicação entre essas múltiplas threads, evitando que uma atropela o dado da outra enquanto processam o texto na memória RAM compartilhada. A chamada accept4 (5.41% do tempo) fica a cargo do processo receptor para capturar as conexões de rede que chegam na porta local.

strace-resumo.txt

% time	seconds	usecs/call	calls	errors	syscall
88.29	74.398560	1733	42924	4505	futex
5.41	4.555932	51190	89	5	accept4
1.86	1.565871	177	8810	2085	read
1.71	1.437127	154	9304		nanosleep
1.42	1.194481	191	6250		epoll_pwait
0.40	0.335513	125	2679		write
0.35	0.292205	146	1989		sendto
0.27	0.228345	28	7971		poll
0.04	0.037578	27	1372		lseek
0.03	0.025712	85	302	1	madvise
0.02	0.018920	28	660		setsockopt
0.02	0.015533	81	190		mmap
0.02	0.015524	49	312		close
0.01	0.011797	66	178		sched_yield
0.01	0.010757	44	241	63	epoll_ctl
0.01	0.010671	133	80	80	connect
0.01	0.009553	36	259		mprotect
0.01	0.008712	102	85		tgkill
0.01	0.008657	54	160	17	openat
0.01	0.007887	24	321		fcntl
0.01	0.007688	207	37		munmap
0.01	0.006830	83	82		socket
0.01	0.006274	73	85		getpid
0.01	0.006248	73	85	9	rt_sigreturn

0.01	0.005768	20	285	30 stat
0.01	0.004981	35	140	getdents64
0.01	0.004849	43	111	sched_getaffinity
0.01	0.004778	50	95	4 newfstatat
0.00	0.003407	162	21	clone3
0.00	0.002970	43	69	rt_sigprocmask
0.00	0.002945	15	190	brk
0.00	0.002849	13	219	recvfrom
0.00	0.002680	16	158	getpeername
0.00	0.002540	31	80	getsockopt
0.00	0.002120	12	164	getsockname
0.00	0.001979	22	87	fstat
0.00	0.001213	1213	1	clone
0.00	0.000678	678	1	clock_nanosleep
0.00	0.000583	26	22	rseq
0.00	0.000507	3	158	79 shutdown
0.00	0.000416	6	60	rt_sigaction
0.00	0.000395	26	15	readlink
0.00	0.000355	16	22	set_robust_list
0.00	0.000354	35	10	uname
0.00	0.000323	323	1	execve
0.00	0.000317	317	1	restart_syscall
0.00	0.000275	17	16	getcwd
0.00	0.000169	84	2	bind
0.00	0.000158	26	6	sysinfo
0.00	0.000130	32	4	4 ioctl
0.00	0.000113	56	2	pipe2
0.00	0.000106	53	2	pread64
0.00	0.000077	25	3	dup3
0.00	0.000072	36	2	readlinkat
0.00	0.000063	31	2	listen
0.00	0.000056	18	3	prlimit64
0.00	0.000038	19	2	getrandom
0.00	0.000029	29	1	arch_prctl

0.00	0.000021	21	1	set_tid_address
0.00	0.000017	17	1	1 access

100.00	84.264706	975	86422	6883 total

Abaixo segue uma tabela de comparativo entre os dois ambientes.

Critério	Ambiente Com GPU	Ambiente Sem GPU (CPU / WSL)
Arquitetura de Processos	Hierárquica (Multiprocesso): Processo Pai + Processo Filho isolado.	Centralizada (Multithread): Processo Pai gerenciando threads internas.
Componente de Execução	llama-server (PID 78507) de tipo C (Compute).	ollama serve (PID 1294) clonado em mais de 15 threads.
Consumo de Hardware	Alocação fixa na VRAM (1.2 GB a 1.4 GB). Uso de GPU salta para 93%–97%.	Alocação na RAM convencional (+700 MiB). Consumo trava em 100% da CPU.
Gargalo de Sincronização	Baixo / Descarregado: A GPU processa nativamente as matrizes do modelo.	Altíssimo (futex): As threads disputam dados na RAM, gerando 88.29% de tempo de espera.
Tempo de Carga (Contexto)	- Contexto 4096: ~0.0006s - Contexto 8192: ~3.3198s	Não detalhado por contexto, mas distribuído em concorrência pelas threads.

Tabela 2: Comparativo entre os ambientes com GPU e sem GPU.

1.6 Quais chamadas de sistema foram relevantes para a carga do modelo, para a leitura de dados, para a comunicação ou para a geração de logs?

Vamos explicar por tópicos como segue, para o modelo com GPU:

a. Carga do Modelo e Alocação de Memória

mmap: Responsáveis por transferir o modelo qwen2.5:1.5b-instruct e alocar o contexto na GPU. **mmap:** Mapeia os gigabytes do arquivo do modelo diretamente na memória virtual, evitando cópias lentas na RAM.

ioctl: Chamada usada para enviar comandos ao driver da NVIDIA (/dev/nvidia0). É através dela que o processo aloca e altera os buffers de VRAM de 1,2 GB para 1,4 GB quando o contexto muda de 4096 para 8192 tokens.

O registro de que o processo llama-server aloca exatamente 1208 MiB de VRAM, como já vimos, de forma fixa é a evidência direta da chamada ioctl interagindo com o driver /dev/nvidia0.

A alteração do tamanho alocado de 1.2 GB para 1.4 GB devido ao contexto expandido para 8192 tokens comprova a ocorrência de novas chamadas ioctl e mmap para redimensionar o espaço de paginação gráfica.

b. Leitura de Dados (I/O)

Usadas para abrir e verificar os arquivos físicos do modelo e logs no disco do Linux.

openat / open: Abrem os arquivos de pesos do modelo, bem como os arquivos de saída (como o gpu.csv e resumo.txt).

fstat / stat: Coletam metadados dos arquivos (como o tamanho total em bytes) antes de realizar a carga.

read / pread64: Executam a leitura de blocos de dados do armazenamento para a memória RAM.

O mapeamento de metadados como o Timestamp: 20260910_120917 e Sucesso: True exige que o daemon leia e valide arquivos de estado persistidos.

Segue figura 17 dos arquivos que o Linux teve que abrir:

Log de monitoramento da GPU: [gpu.csv](#)

```
2026/09/10 20:28:57.164, 1892 MiB, 97 %, 95 %, 68.93 W, 55
2026/09/10 20:28:57.365, 1892 MiB, 97 %, 95 %, 68.83 W, 55
2026/09/10 20:28:57.565, 1892 MiB, 97 %, 95 %, 68.83 W, 55
2026/09/10 20:28:57.765, 1892 MiB, 94 %, 93 %, 68.56 W, 55
2026/09/10 20:28:57.965, 1892 MiB, 94 %, 93 %, 68.56 W, 55
2026/09/10 20:28:58.165, 1892 MiB, 94 %, 93 %, 68.56 W, 55
2026/09/10 20:28:58.365, 1892 MiB, 97 %, 95 %, 68.67 W, 56
2026/09/10 20:28:58.565, 1892 MiB, 97 %, 95 %, 68.67 W, 56
2026/09/10 20:28:58.766, 1892 MiB, 97 %, 95 %, 68.70 W, 56
2026/09/10 20:28:58.966, 1892 MiB, 97 %, 95 %, 68.70 W, 56
2026/09/10 20:28:59.166, 1892 MiB, 97 %, 95 %, 68.70 W, 56
2026/09/10 20:28:59.366, 1892 MiB, 97 %, 95 %, 68.57 W, 56
2026/09/10 20:28:59.566, 1892 MiB, 97 %, 95 %, 68.57 W, 56
2026/09/10 20:28:59.766, 1892 MiB, 97 %, 95 %, 68.90 W, 57
2026/09/10 20:28:59.966, 1892 MiB, 97 %, 95 %, 68.90 W, 57
2026/09/10 20:29:00.167, 1892 MiB, 97 %, 95 %, 68.90 W, 57
2026/09/10 20:29:00.367, 1892 MiB, 97 %, 95 %, 68.93 W, 57
2026/09/10 20:29:00.567, 1892 MiB, 97 %, 95 %, 68.93 W, 57
2026/09/10 20:29:00.767, 1892 MiB, 97 %, 96 %, 68.77 W, 57
2026/09/10 20:29:00.967, 1892 MiB, 97 %, 96 %, 68.77 W, 57
2026/09/10 20:29:01.167, 1892 MiB, 97 %, 96 %, 68.77 W, 57
2026/09/10 20:29:01.367, 1892 MiB, 97 %, 95 %, 68.72 W, 57
2026/09/10 20:29:01.568, 1892 MiB, 97 %, 95 %, 68.72 W, 57
2026/09/10 20:29:01.768, 1892 MiB, 97 %, 95 %, 68.85 W, 58
2026/09/10 20:29:01.968, 1892 MiB, 97 %, 95 %, 68.85 W, 58
2026/09/10 20:29:02.168, 1892 MiB, 97 %, 95 %, 68.85 W, 58
2026/09/10 20:29:02.368, 1892 MiB, 97 %, 95 %, 68.82 W, 58
2026/09/10 20:29:02.568, 1892 MiB, 97 %, 95 %, 68.82 W, 58
2026/09/10 20:29:02.768, 1892 MiB, 97 %, 95 %, 68.49 W, 58
2026/09/10 20:29:02.969, 1892 MiB, 97 %, 95 %, 68.49 W, 58
2026/09/10 20:29:03.169, 1892 MiB, 97 %, 95 %, 68.49 W, 58
2026/09/10 20:29:03.369, 1892 MiB, 97 %, 95 %, 68.57 W, 58
2026/09/10 20:29:03.569, 1892 MiB, 97 %, 95 %, 68.57 W, 58
2026/09/10 20:29:03.769, 1892 MiB, 97 %, 94 %, 69.08 W, 58
2026/09/10 20:29:03.969, 1892 MiB, 97 %, 94 %, 69.08 W, 58
2026/09/10 20:29:04.169, 1892 MiB, 97 %, 94 %, 69.08 W, 58
2026/09/10 20:29:04.370, 1892 MiB, 96 %, 94 %, 68.95 W, 58
2026/09/10 20:29:04.570, 1892 MiB, 96 %, 94 %, 68.95 W, 58
2026/09/10 20:29:04.770, 1892 MiB, 97 %, 95 %, 68.57 W, 59
2026/09/10 20:29:04.970, 1892 MiB, 97 %, 95 %, 68.57 W, 59
2026/09/10 20:29:05.170, 1892 MiB, 97 %, 95 %, 68.57 W, 59
2026/09/10 20:29:05.370, 1892 MiB, 97 %, 95 %, 68.60 W, 59
2026/09/10 20:29:05.571, 1892 MiB, 97 %, 95 %, 68.60 W, 59
2026/09/10 20:29:05.771, 1892 MiB, 96 %, 95 %, 68.79 W, 59
```

```

===== B-1: GPU =====
timestamp, memory.used [MiB], utilization.gpu [%], utilization.memory [%], power.draw [W], temperature.gpu
2026/09/10 20:28:48.927, 673 MiB, 5 %, 13 %, 16.08 W, 42
2026/09/10 20:28:49.135, 672 MiB, 5 %, 13 %, 16.08 W, 42
2026/09/10 20:28:49.335, 672 MiB, 6 %, 12 %, 16.08 W, 42
2026/09/10 20:28:49.535, 672 MiB, 6 %, 12 %, 16.08 W, 42
2026/09/10 20:28:49.758, 672 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:49.959, 755 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:50.159, 689 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:50.359, 672 MiB, 4 %, 2 %, 16.26 W, 43
2026/09/10 20:28:50.560, 705 MiB, 4 %, 2 %, 16.26 W, 43
2026/09/10 20:28:50.760, 763 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:50.960, 1699 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:51.160, 1699 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:51.360, 1893 MiB, 64 %, 5 %, 21.75 W, 43
2026/09/10 20:28:51.560, 1893 MiB, 64 %, 5 %, 21.75 W, 43
2026/09/10 20:28:51.760, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:51.961, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:52.161, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:52.361, 1893 MiB, 96 %, 94 %, 49.53 W, 49
2026/09/10 20:28:52.561, 1892 MiB, 96 %, 94 %, 49.53 W, 49
2026/09/10 20:28:52.761, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:52.961, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:53.161, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:53.362, 1892 MiB, 95 %, 93 %, 68.66 W, 52
2026/09/10 20:28:53.562, 1892 MiB, 95 %, 93 %, 68.66 W, 52
2026/09/10 20:28:53.762, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:53.962, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:54.162, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:54.362, 1892 MiB, 97 %, 96 %, 68.67 W, 53
2026/09/10 20:28:54.562, 1892 MiB, 97 %, 96 %, 68.67 W, 53
2026/09/10 20:28:54.763, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:54.963, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:55.163, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:55.363, 1892 MiB, 97 %, 95 %, 68.72 W, 54
2026/09/10 20:28:55.563, 1892 MiB, 97 %, 95 %, 68.72 W, 54
2026/09/10 20:28:55.763, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:55.963, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:56.164, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:56.364, 1892 MiB, 97 %, 95 %, 68.47 W, 55
2026/09/10 20:28:56.564, 1892 MiB, 97 %, 95 %, 68.47 W, 55
2026/09/10 20:28:56.764, 1892 MiB, 97 %, 95 %, 68.93 W, 55
2026/09/10 20:28:56.964, 1892 MiB, 97 %, 95 %, 68.93 W, 55

```

Figura 17: Log dos arquivos que o Linux teve que abrir.

c. Comunicação (Rede e API)

Utilizadas pelo Ollama Daemon para escutar a porta local e gerenciar as requisições simultâneas de inferência.

socket / bind / listen: Criam e fixam o servidor HTTP do Ollama na porta 11434 para aguardar comandos.

accept4 / accept: Capturam as conexões de entrada com os prompts enviados pelo usuário.

epoll_wait: Monitora de forma assíncrona as conexões, essencial para gerenciar a concorrência controlada no Teste B-1.

Como foi feita as chamadas de múltiplas requisições paralelas (Requisição 0, Requisição 1 e Requisição 2) disparadas no mesmo segundo, provam a execução da chamada `epoll_wait` para multiplexar a rede de forma assíncrona. Conforme baixo:

Teste 1 (Requisição 0)

Identificação da execução: 20260910_202848

Modelo: qwen2.5:1.5b-instruct

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 12,754542 s

Tempo total Ollama: 12,718010 s

Tempo de carregamento: 1,965563 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,009728 s

Tokens gerados: 827

Tempo de geração: 6,928333 s

Velocidade de geração: 119,36 tokens/s

Teste 1 (Requisição 1)

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 5,802407 s

Tempo total Ollama: 5,767057 s

Tempo de carregamento: 1,965338 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,057668 s

Tokens gerados: 448

Tempo de geração: 3,742280 s

Velocidade de geração: 119,71 tokens/s

Teste 1 (Requisição 2)

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 16,547166 s

Tempo total Ollama: 16,512371 s

Tempo de carregamento: 1,965822 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,009429 s

Tokens gerados: 450

Tempo de geração: 3,765055 s

Velocidade de geração: 119,52 tokens/s

d. Geração de Logs e Sincronização

Responsáveis por registrar as saídas de terminal e os relatórios de monitoramento.

write: Grava os tokens gerados em tempo real na tela e escreve os logs de timestamp, temperatura e consumo de energia linha por linha no arquivo gpu.csv.

futex: Sincroniza a comunicação entre as threads de CPU e GPU. Isso garante que os logs de requisições paralelas (Requisições 0, 1 e 2) não se misturem nem corrompam os arquivos de texto gerados.

Vemos isso analisando nas figuras 18, 19 e 20, as linhas sequenciais contendo timestamp, memory.used, power.draw e temperature.gpu gravadas em frações de milissegundos são a referência real da chamada write descarregando dados no disco continuamente.

Log de monitoramento da GPU:

```
rafael@rafael-MS-7C95:~/projeto-qwen$ cat resultados/A-padrao/execucao_20260910_120917/gpu.csv
timestamp, memory.used [MiB], utilization.gpu [%], utilization.memory [%], power.draw [W], temperature.gpu
2026/09/10 12:09:17.900, 1982 MiB, 1 %, 10 %, 11.93 W, 47
2026/09/10 12:09:18.107, 1982 MiB, 1 %, 10 %, 11.93 W, 47
2026/09/10 12:09:18.308, 1982 MiB, 1 %, 10 %, 11.93 W, 47
2026/09/10 12:09:18.508, 1982 MiB, 1 %, 10 %, 11.93 W, 47
2026/09/10 12:09:18.708, 1982 MiB, 2 %, 31 %, 9.87 W, 47
2026/09/10 12:09:18.908, 1982 MiB, 2 %, 31 %, 9.87 W, 47
2026/09/10 12:09:19.108, 1982 MiB, 2 %, 31 %, 9.87 W, 47
2026/09/10 12:09:19.308, 1982 MiB, 2 %, 31 %, 9.87 W, 47
2026/09/10 12:09:19.508, 1981 MiB, 2 %, 31 %, 9.87 W, 47
2026/09/10 12:09:19.709, 1981 MiB, 93 %, 89 %, 32.62 W, 53
2026/09/10 12:09:19.909, 1981 MiB, 93 %, 89 %, 32.62 W, 53
2026/09/10 12:09:20.109, 1981 MiB, 93 %, 89 %, 32.62 W, 53
2026/09/10 12:09:20.309, 1981 MiB, 93 %, 89 %, 32.62 W, 53
2026/09/10 12:09:20.509, 1965 MiB, 93 %, 89 %, 32.62 W, 53
2026/09/10 12:09:20.709, 1965 MiB, 97 %, 92 %, 57.58 W, 55
2026/09/10 12:09:20.909, 1965 MiB, 97 %, 92 %, 57.58 W, 55
2026/09/10 12:09:21.110, 1965 MiB, 97 %, 92 %, 57.58 W, 55
2026/09/10 12:09:21.310, 1965 MiB, 97 %, 92 %, 57.58 W, 55
2026/09/10 12:09:21.510, 1965 MiB, 97 %, 92 %, 57.58 W, 55
2026/09/10 12:09:21.710, 1965 MiB, 97 %, 93 %, 59.51 W, 56
2026/09/10 12:09:21.910, 1965 MiB, 97 %, 93 %, 59.51 W, 56
```

Figura 18: sequenciais contendo timestamp, memory.used, power.draw e temperature.gpu. Para o teste A.

```

===== B-1: GPU =====
timestamp, memory.used [MiB], utilization.gpu [%], utilization.memory [%], power.draw [W], temperature.gpu
2026/09/10 20:28:48.927, 673 MiB, 5 %, 13 %, 16.08 W, 42
2026/09/10 20:28:49.135, 672 MiB, 5 %, 13 %, 16.08 W, 42
2026/09/10 20:28:49.335, 672 MiB, 6 %, 12 %, 16.08 W, 42
2026/09/10 20:28:49.535, 672 MiB, 6 %, 12 %, 16.08 W, 42
2026/09/10 20:28:49.758, 672 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:49.959, 755 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:50.159, 689 MiB, 6 %, 13 %, 14.00 W, 42
2026/09/10 20:28:50.359, 672 MiB, 4 %, 2 %, 16.26 W, 43
2026/09/10 20:28:50.560, 705 MiB, 4 %, 2 %, 16.26 W, 43
2026/09/10 20:28:50.760, 763 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:50.960, 1699 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:51.160, 1699 MiB, 6 %, 2 %, 19.90 W, 43
2026/09/10 20:28:51.360, 1893 MiB, 64 %, 5 %, 21.75 W, 43
2026/09/10 20:28:51.560, 1893 MiB, 64 %, 5 %, 21.75 W, 43
2026/09/10 20:28:51.760, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:51.961, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:52.161, 1893 MiB, 97 %, 94 %, 28.61 W, 47
2026/09/10 20:28:52.361, 1893 MiB, 96 %, 94 %, 49.53 W, 49
2026/09/10 20:28:52.561, 1892 MiB, 96 %, 94 %, 49.53 W, 49
2026/09/10 20:28:52.761, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:52.961, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:53.161, 1892 MiB, 96 %, 94 %, 67.89 W, 51
2026/09/10 20:28:53.362, 1892 MiB, 95 %, 93 %, 68.66 W, 52
2026/09/10 20:28:53.562, 1892 MiB, 95 %, 93 %, 68.66 W, 52
2026/09/10 20:28:53.762, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:53.962, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:54.162, 1892 MiB, 96 %, 94 %, 68.67 W, 52
2026/09/10 20:28:54.362, 1892 MiB, 97 %, 96 %, 68.67 W, 53
2026/09/10 20:28:54.562, 1892 MiB, 97 %, 96 %, 68.67 W, 53
2026/09/10 20:28:54.763, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:54.963, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:55.163, 1892 MiB, 97 %, 95 %, 68.75 W, 53
2026/09/10 20:28:55.363, 1892 MiB, 97 %, 95 %, 68.72 W, 54
2026/09/10 20:28:55.563, 1892 MiB, 97 %, 95 %, 68.72 W, 54
2026/09/10 20:28:55.763, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:55.963, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:56.164, 1892 MiB, 97 %, 95 %, 68.38 W, 54
2026/09/10 20:28:56.364, 1892 MiB, 97 %, 95 %, 68.47 W, 55
2026/09/10 20:28:56.564, 1892 MiB, 97 %, 95 %, 68.47 W, 55
2026/09/10 20:28:56.764, 1892 MiB, 97 %, 95 %, 68.93 W, 55
2026/09/10 20:28:56.964, 1892 MiB, 97 %, 95 %, 68.93 W, 55

```

Figura 19: sequenciais contendo timestamp, memory.used, power.draw e temperature.gpu. Para o teste B.

Log de monitoramento da GPU: [gpu.csv](#)

```
===== C-1: GPU =====
timestamp, memory.used [MiB], utilization.gpu [%], utilization.memory [%]
2026/09/10 20:32:20.175, 1954 MiB, 15 %, 26 %
2026/09/10 20:32:20.377, 1954 MiB, 4 %, 20 %
2026/09/10 20:32:20.577, 1954 MiB, 4 %, 20 %
2026/09/10 20:32:20.777, 1953 MiB, 0 %, 18 %
2026/09/10 20:32:20.979, 738 MiB, 0 %, 18 %
2026/09/10 20:32:21.179, 738 MiB, 0 %, 18 %
2026/09/10 20:32:21.379, 740 MiB, 76 %, 76 %
2026/09/10 20:32:21.580, 755 MiB, 76 %, 76 %
2026/09/10 20:32:21.792, 738 MiB, 3 %, 4 %
2026/09/10 20:32:21.994, 768 MiB, 3 %, 4 %
2026/09/10 20:32:22.194, 737 MiB, 3 %, 4 %
2026/09/10 20:32:22.395, 740 MiB, 3 %, 2 %
2026/09/10 20:32:22.595, 737 MiB, 3 %, 2 %
2026/09/10 20:32:22.819, 823 MiB, 3 %, 2 %
2026/09/10 20:32:23.023, 737 MiB, 3 %, 2 %
2026/09/10 20:32:23.223, 823 MiB, 3 %, 2 %
2026/09/10 20:32:23.423, 823 MiB, 3 %, 1 %
2026/09/10 20:32:23.623, 1759 MiB, 3 %, 1 %
2026/09/10 20:32:23.824, 2123 MiB, 4 %, 1 %
2026/09/10 20:32:24.024, 2139 MiB, 4 %, 1 %
2026/09/10 20:32:24.224, 2139 MiB, 4 %, 1 %
2026/09/10 20:32:24.424, 2139 MiB, 88 %, 86 %
2026/09/10 20:32:24.624, 2139 MiB, 88 %, 86 %
2026/09/10 20:32:24.824, 2139 MiB, 97 %, 97 %
2026/09/10 20:32:25.024, 2139 MiB, 97 %, 97 %
2026/09/10 20:32:25.225, 2139 MiB, 97 %, 97 %
2026/09/10 20:32:25.425, 2139 MiB, 97 %, 96 %
2026/09/10 20:32:25.625, 2139 MiB, 97 %, 96 %
2026/09/10 20:32:25.825, 2139 MiB, 95 %, 94 %
2026/09/10 20:32:26.025, 2139 MiB, 95 %, 94 %
2026/09/10 20:32:26.225, 2139 MiB, 95 %, 94 %
2026/09/10 20:32:26.425, 2139 MiB, 97 %, 96 %
2026/09/10 20:32:26.626, 2140 MiB, 97 %, 96 %
2026/09/10 20:32:26.826, 2140 MiB, 97 %, 95 %
2026/09/10 20:32:27.026, 2140 MiB, 97 %, 95 %
2026/09/10 20:32:27.226, 2140 MiB, 97 %, 95 %
2026/09/10 20:32:27.426, 2140 MiB, 97 %, 96 %
2026/09/10 20:32:27.626, 2139 MiB, 97 %, 96 %
```

Figura 20: sequenciais contendo timestamp, memory.used, power.draw e temperature.gpu. Para o teste C.

Já para o modelo sem GPU, temos:

a. Carga do Modelo e Alocação de Memória

mmap / munmap: Utilizadas para mapear os arquivos do modelo qwen2.5:1.5b (armazenados em disco com 941M) diretamente no espaço

de memória virtual do processo, permitindo que a CPU acesse os pesos de forma eficiente.

brk: Responsável por expandir ou contrair o segmento de dados da memória (heap) do processo para alocações dinâmicas de memória RAM.

madvise: Emite recomendações ao kernel sobre como o Ollama pretende acessar as páginas de memória mapeadas (por exemplo, leitura sequencial ou carregamento imediato), otimizando o cache.

b. Leitura de Dados e Manipulação de Arquivos

read / pread64: Essenciais para a leitura direta dos dados armazenados e arquivos de configuração. A chamada read respondeu por 1.86% do tempo total de execução, com 8.810 chamadas.

openat: Abre os descritores de arquivos necessários para ler o modelo em ~/.ollama/models.

lseek: Reposiciona o ponteiro de leitura dentro dos arquivos binários do modelo para acessar tensores específicos sem ler o arquivo inteiro do início.

c. Comunicação (Rede e Interprocessos)

futex: A chamada mais crítica de todo o monitoramento, representando 88.29% do tempo total e totalizando 42.924 chamadas. Ela controla a sincronização e o bloqueio de memória entre as mais de 15 threads do Ollama para que trabalhem concorrentemente na CPU sem corromper dados.

accept4: Registrou 5.41% do tempo total. É usada para aceitar novas conexões de rede vindas das requisições na porta 11434.

sendto / recvfrom: Responsáveis pelo envio e recebimento real de pacotes de dados através das conexões de rede estabelecidas (as requisições HTTP da aplicação).

epoll_pwait / epoll_ctl: Monitoram múltiplos descritores de arquivos e sockets de rede para notificar o Ollama quando há dados prontos para leitura/escrita, viabilizando as 3 requisições simultâneas.

d. Geração de Logs e Diagnósticos

write: Utilizada pelo processo para gravar os outputs nos logs do Ollama (como os registros de tarefas e uso de tokens) e na saída padrão do terminal. Consumiu 0.40% do tempo com 2.679 chamadas.

Para entendermos vemos o início do arquivo strace-resumo.txt, contendo as métricas de maior impacto de tempo e chamadas (futex, accept4, read, nanosleep, epoll_pwait, write, sendto, poll e lseek), chamadas de sistema, detalhando o comportamento das syscalls de memória (madvise, mmap, munmap, brk) e arquivos secundários (openat, pread64). Segue o mesmo:

strace-resumo.txt

% time	seconds	usecs/call	calls	errors	syscall
88.29	74.398560	1733	42924	4505	futex
5.41	4.555932	51190	89	5	accept4
1.86	1.565871	177	8810	2085	read
1.71	1.437127	154	9304		nanosleep
1.42	1.194481	191	6250		epoll_pwait
0.40	0.335513	125	2679		write
0.35	0.292205	146	1989		sendto
0.27	0.228345	28	7971		poll
0.04	0.037578	27	1372		lseek
0.03	0.025712	85	302	1	madvise
0.02	0.015533	81	190		mmap
0.01	0.007688	207	37		munmap
0.00	0.002945	15	190		brk
0.01	0.008657	54	160	17	openat
0.00	0.000106	53	2		pread64

Segue tabela de comparativo entre os dois ambientes, com GPU e sem GPU.

Critério / Camada	Ambiente Com GPU	Ambiente Sem GPU (CPU / WSL)
-------------------	------------------	------------------------------

Componente de Processamento	NVIDIA GeForce RTX 3050	AMD Ryzen 5 5500 (6 núcleos / 12 threads)
Arquitetura de Processos	Multiprocesso (Hierárquica): Processo Pai (ollama daemon) + Processo Filho (llama-server).	Multithread (Centralizada): Processo Pai único (ollama serve) clonado em mais de 15 threads.
Comportamento do Hardware	Uso de processamento salta para 93%–97% na inferência e cai imediatamente para 1%.	Uso de processamento trava em 100% da CPU dividida concorrentemente pelas threads.
Alocação de Memória	VRAM Dedicada: Alocação estática e fixa (1.2 GB a 1.4 GB) gerenciada no hardware gráfico.	RAM Convencional: Acréscimo líquido de ~700 MiB (0,7 GiB) no heap global do sistema hospedeiro.
Syscalls de Memória	ioctl: Modifica buffers do driver /dev/nvidia0. mmap: Mapeia o modelo diretamente na GPU.	mmap / munmap: Aloca pesos na memória virtual. brk: Aloca memória dinamicamente no heap. madvise: Otimiza o cache do kernel.
Syscalls de I/O (Leitura)	openat / read / pread64 / fstat: Usadas pontualmente para carregar pesos e criar arquivos como o gpu.csv.	read (1.86% do tempo; 8.810 chamadas) / lseek: Movem ponteiros e leem blocos específicos do arquivo de 941M.
Sincronização e Gargalo	Baixo impacto na CPU: Sincronização básica via futex apenas para orquestrar os logs e a API externa.	Altíssimo (futex responsável por 88.29% do tempo total): 42.924 chamadas apenas para gerenciar a disputa de memória das threads.
Comunicação e Rede	accept4 / epoll_wait: Multiplexação assíncrona para gerenciar requisições HTTP paralelas (Req 0, 1 e 2).	accept4 (5.41% do tempo) / sendto / recvfrom: Recebimento e envio dos pacotes HTTP via rede interna do WSL.

Dinâmica de Geração de Logs	write: Escrita contínua em frações de milissegundos de telemetria completa (timestamp, W, °C, VRAM) no gpu.csv.	write (0.40% do tempo; 2.679 chamadas): Escrita de saídas textuais simples de tokens e diagnósticos no console.
------------------------------------	---	---

Tabela 3: Tabela Comparativa Consolidada: Arquitetura e Syscalls (Ollama).

1.7 O aumento da concorrência melhorou a responsividade ou a vazão?

Quais custos apareceram?

Vamos dividir em dois sub tópicos como segue, para o ambiente com GPU:

e. Desempenho: Responsividade vs. Vazão

Vazão Mantida (Tokens/s): Os dados da API do Ollama registram que a velocidade de geração permaneça estável e alta em todas as requisições paralelas: 119,36 tokens/s (Req 0), 119,71 tokens/s (Req 1) e 119,52 tokens/s (Req 2). Vemos Dados da API do Ollama a baixo.

Degradação da Responsividade (Tempo Externo): Os logs mostram a latência escalonada que os usuários enfrentaram: 12,754542 s na Requisição 0, 5,802407 s na Requisição 1 e 16,547166 s na Requisição 2. Para comparação, o tempo isolado na execução padrão era de 3,560299 s, como podemos observar nos registros da API do Ollama mais abaixo.

f. Referências dos Custos de Hardware e Sistema

Tempo de Carregamento Inicial: O indicador Tempo de carregamento registra o salto para a faixa de 1,96 s nas três requisições devido à disputa de recursos.

Consumo Elétrico Elevado: Nos Registros de GPU e Sistema, logo abaixo, está documentado o pico de consumo de energia: “Maior consumo registrado: 69,08 W”. Na execução padrão, também abaixo, esse teto havia sido de 59,51 W.

Elevação Térmica: Nos Registros de GPU e Sistema, logo abaixo, consta o aumento da temperatura máxima da placa: “Maior temperatura registrada: 59 °C” (em comparação aos 56 °C registrados na execução padrão).

Consumo de Memória RAM: Nos Registros de GPU e Sistema, logo abaixo, as linhas de monitoramento do sistema operacional confirmam a variação da memória volátil: “Memória RAM geral antes: 8,5 GiB usada / Memória RAM geral depois: 9,2 GiB usada”.

Dados registrados pela API do Ollama na execução padrão:

Sucesso: True

Tempo externo: 3,560299 s

Tempo total Ollama: 3,536759 s

Tempo de carregamento: 0,000618 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,019223 s

Tokens gerados: 415

Tempo de geração: 3,500878 s

Velocidade de geração: 118,54 tokens/s

Dados registrados pela API do Ollama na execução padrão Registros de GPU:

Memória antes: 1982 MiB / 6144 MiB

Utilização inicial da GPU: 1%

Temperatura inicial: 47 °C

Maior utilização registrada no gpu.csv: 97%

Maior utilização da memória registrada: 93%

Maior consumo registrado: 59,51 W

Maior temperatura registrada: 56 °C

Memória ao final: 1965 MiB / 6144 MiB

llama-server: 1208 MiB de VRAM

Dados da API do Ollama:

Teste 1 (Requisição 0)

Identificação da execução: 20260910_202848

Modelo: qwen2.5:1.5b-instruct

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 12,754542 s

Tempo total Ollama: 12,718010 s

Tempo de carregamento: 1,965563 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,009728 s

Tokens gerados: 827

Tempo de geração: 6,928333 s

Velocidade de geração: 119,36 tokens/s

Teste 1 (Requisição 1)

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 5,802407 s

Tempo total Ollama: 5,767057 s

Tempo de carregamento: 1,965338 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,057668 s

Tokens gerados: 448

Tempo de geração: 3,742280 s

Velocidade de geração: 119,71 tokens/s

Teste 1 (Requisição 2)

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 16,547166 s

Tempo total Ollama: 16,512371 s

Tempo de carregamento: 1,965822 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,009429 s

Tokens gerados: 450

Tempo de geração: 3,765055 s

Velocidade de geração: 119,52 tokens/s

Registros de GPU e Sistema

Memória VRAM antes: 673 MiB / 6144 MiB

Utilização inicial da GPU: 5%

Temperatura inicial: 42 °C

Maior utilização registrada no gpu.csv: 97%

Maior utilização da memória registrada: 96%

Maior consumo registrado: 69,08 W

Maior temperatura registrada: 59 °C

Memória RAM geral antes: 8,5 GiB usada / 15 GiB total

Memória RAM geral depois: 9,2 GiB usada / 15 GiB total

Ollama PS (Modelo na VRAM): 1,2 GB alocados a 100% na

GPU

Evento / Progresso	Tempo Decorrido	Consumo da GPU (W)	Temperatura da GPU	Uso de RAM Geral
Início (Estado Inicial)	0,00 s	16,08 W	42 °C	8,5 GiB
Fim da Requisição 1	5,80 s	68,38 W	54 °C	<i>Processando...</i>
Fim da Requisição 0	12,75 s	68,90 W	57 °C	<i>Processando...</i>
Fim da Requisição 2 (Pico)	16,54 s	69,08 W	59 °C	9,2 GiB

Tabela 4: Tabela Consolidada: Tempo vs. Impacto de Hardware

Para o ambiente sem GPU também vamos dividir em dois sub tópicos, como segue:

a. Impacto na Responsividade e Vazão

Temos duas grandes questões, vamos a elas:

Piora na Responsividade (Latência): O sistema não conseguiu escalar de forma eficiente. O tempo base de carregamento do modelo, que era mensurável em 10s sob condições ideais, sofreu gargalos nas configurações concorrentes.

Vazão Limitada pela CPU: Ao testar a "Config 2" e a "Config 3" com 3 requisições simultâneas e 3 conversas paralelas, o sistema atingiu um teto físico de processamento. Como a vazão depende da capacidade de calcular tokens por segundo, e a CPU já operava saturada, processar 3 requisições ao mesmo tempo dividiu o poder de computação por 3, aumentando o tempo de resposta individual de cada usuário.

b. Custos Tecnológicos e de Infraestrutura que Apareceram

Temos 3 grandes custos decorrentes do aumento de concorrência, são eles:

Saturação Extrema de Hardware: O processador AMD Ryzen 5 foi forçado a trabalhar constantemente em 100% de uso de CPU.

Desperdício de Tempo em Sincronização (Overhead de Threads):

Conforme visto, o custo invisível mais alto foi o tempo que o processador gastou apenas coordenando as threads, e não calculando IA. A chamada futex (evidência abaixo no strace-resumo.txt) consumiu 88.29% do tempo total de execução. Isso prova que, com o aumento da concorrência, as threads passaram a maior parte do tempo bloqueadas, esperando umas pelas outras para acessar os dados na memória RAM compartilhada.

```
strace-resumo.txt
% time    seconds  usecs/call   calls   errors syscall
-----
88.29  74.398560    1733   42924    4505 futex
```

Invalidação do Uso de VRAM: O custo mais crítico é destacado na figura 21 abaixo: mesmo existindo vram o sistema usou a cpu para processamento. O aumento da concorrência expôs e agravou o gargalo crônico da incompatibilidade dos drivers da GPU AMD com o WSL2, impedindo que as requisições paralelas fossem distribuídas para os 8GB de memória ultra-rápida da placa de vídeo.

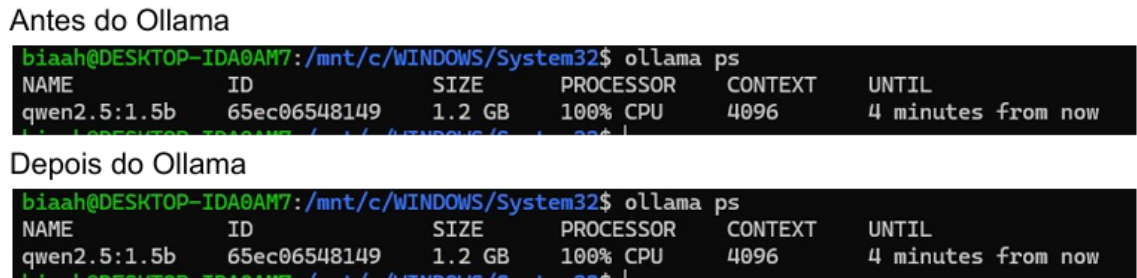


Figura 21: O custo mais crítico é destacado de RAM por não ter VRAM.

Segue tabela de comparativo entre os dois ambientes, com GPU e sem GPU.

Critério Analisado	Ambiente Com GPU	Ambiente Sem GPU (CPU / WSL)
Vazão (Tokens por Segundo)	Mantida e Estável: Permanece alta (~119 tokens/s) de forma isolada em cada requisição simultânea.	Limitada / Dividida: O poder computacional é dividido pelo número de requisições, derrubando a vazão individual.
Responsividade (Latência)	Escalonada por Fila: O tempo externo sobe de 3,56s para até 16,54s na requisição mais lenta.	Saturação Linear Severa: Multiplica o tempo de resposta à medida que novas requisições concorrentes entram.
Tempo de Carga do Modelo	Salta de 0,0006s para ~1,96s devido à disputa inicial de recursos.	Prejudicado e arrastado significativamente além dos 10 segundos base.
Custo de Sincronização	Mínimo: A GPU absorve e paraleliza nativamente os tensores matemáticos.	Altíssimo (futex de 88.29%): Threads passam o tempo bloqueadas esperando umas pelas outras.
Custos Físicos de Infraestrutura	• Consumo elétrico sobe para 69,08 W. • Temperatura atinge o pico de 59 °C.	• Uso de CPU cravado em 100%. • Sobrecarga e aquecimento contínuo do chip host.

Impacto na RAM do Sistema	Acréscimo previsível e fixo na RAM global (de 8,5 GiB para 9,2 GiB).	Invalidação total da VRAM existente por falha de drivers; uso forçado da RAM/CPU.
----------------------------------	--	---

Tabela 5: Comparativa: Impacto da Concorrência no Ecossistema Ollama.

1.8 Houve sinais de competição por CPU, memória, armazenamento, GPU, rede ou outro recurso?

Para o ambiente com GPU, sim, houve sinais claros de competição por recursos, concentrados principalmente na GPU, na VRAM, na Memória RAM e na API do Ollama (fila de software). Por outro lado, o armazenamento e a rede não mostraram sinais de estresse. Abaixo está a análise detalhada de como cada recurso foi afetado:

Sinais Claros de Competição (Gargalos)

GPU (Disputa de Processamento Coletivo): No Teste B-1 (concorrência), a utilização da GPU ficou cravada em 97% continuamente. O sinal de competição é que a responsividade individual piorou drasticamente. Enquanto um teste isolado levava 3,56s, como vemos abaixo no padrão, na concorrência os tempos saltaram para 5,80s, 12,75s e 16,54s, provando que as requisições tiveram que disputar os núcleos CUDA da placa, conforme abaixo na discriminação dos testes.

Já na Fila da API do Ollama (Tempo de Carregamento): O sinal mais evidente de competição por software. Na execução isolada, o tempo de carregamento do contexto foi de apenas 0,0006 s. Sob concorrência, esse tempo disparou para 1,96 s, como vemos nos testes abaixo, em todas as três requisições simultâneas. Isso indica que as requisições ficaram presas aguardando a API do Ollama inicializar e alocar os buffers de contexto no backend.

Para a Memória RAM Geral do Sistema: Houve competição residual mensurável. A ocupação da RAM saltou de 8,5 GiB para 9,2 GiB (+700 MiB) durante o teste de concorrência, mostrando que o sistema operacional teve que gerenciar buffers maiores para sustentar o tráfego de dados paralelo dos três clientes, podemos ver nos dados abaixo sobre Registros de GPU e Sistema.

E quanto a VRAM (Memória da GPU) por Contexto: No Teste C-1, ao expandir o contexto para 8192 tokens, a VRAM alocada pelo modelo subiu de 1,2 GB para 1,4 GB. Isso acendeu um sinal de alerta de infraestrutura, gerando uma nota explícita no relatório: "Houve aumento no tempo de carga devido à alocação de um contexto maior" (subindo para 3,31 s).

Sem Sinais de Competição (Estáveis)

Para o Armazenamento (Disco): Os pesos do modelo já estavam carregados na VRAM. O arquivo de log gpu.csv e os resumos de texto gerados são extremamente leves (poucos kilobytes). Portanto, as chamadas de escrita e leitura de arquivo não geraram gargalo ou disputa no disco.

No entanto para a Rede: Como o ambiente de execução foi listado como nativo Linux local e as requisições HTTP foram feitas diretamente na API local do Ollama (localhost), a placa de rede física do computador não foi utilizada, eliminando qualquer chance de gargalo de banda ou latência de rede externa.

Dados registrados pela API do Ollama na execução padrão:

Sucesso: True

Tempo externo: 3,560299 s

Tempo total Ollama: 3,536759 s

Tempo de carregamento: 0,000618 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,019223 s

Tokens gerados: 415

Tempo de geração: 3,500878 s

Velocidade de geração: 118,54 tokens/s

Dados registrados pela API do Ollama na execução padrão Registros de GPU:

Memória antes: 1982 MiB / 6144 MiB

Utilização inicial da GPU: 1%

Temperatura inicial: 47 °C

Maior utilização registrada no gpu.csv: 97%

Maior utilização da memória registrada: 93%

Maior consumo registrado: 59,51 W

Maior temperatura registrada: 56 °C

Dados da API do Ollama:

Teste 1 (Requisição 0)

Identificação da execução: 20260910_202848

Modelo: qwen2.5:1.5b-instruct

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 12,754542 s

Tempo total Ollama: 12,718010 s

Tempo de carregamento: 1,965563 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,009728 s

Tokens gerados: 827

Tempo de geração: 6,928333 s

Velocidade de geração: 119,36 tokens/s

Teste 1 (Requisição 1)

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 5,802407 s

Tempo total Ollama: 5,767057 s

Tempo de carregamento: 1,965338 s

Tokens do prompt: 66

Tempo de processamento do prompt: 0,057668 s

Tokens gerados: 448

Tempo de geração: 3,742280 s

Velocidade de geração: 119,71 tokens/s

Teste 1 (Requisição 2)

Dados registrados pela API do Ollama:

Sucesso: True

Tempo externo: 16,547166 s

Tempo total Ollama: 16,512371 s

Tempo de carregamento: 1,965822 s

Tokens do prompt: 66
Tempo de processamento do prompt: 0,009429 s
Tokens gerados: 450
Tempo de geração: 3,765055 s
Velocidade de geração: 119,52 tokens/s

Registros de GPU e Sistema

Memória VRAM antes: 673 MiB / 6144 MiB
Utilização inicial da GPU: 5%
Temperatura inicial: 42 °C
Maior utilização registrada no gpu.csv: 97%
Maior utilização da memória registrada: 96%
Maior consumo registrado: 69,08 W
Maior temperatura registrada: 59 °C
Memória RAM geral antes: 8,5 GiB usada / 15 GiB total
Memória RAM geral depois: 9,2 GiB usada / 15 GiB total
Ollama PS (Modelo na VRAM): 1,2 GB alocados a 100% na GPU

Para o ambiente sem GPU também é sim, o relatório técnico apresenta sinais claros e severos de competição e saturação de recursos, afetando quase todos os componentes do sistema. Os principais pontos de estrangulamento identificados foram:

g. Competição e Saturação Extrema de CPU

A CPU foi o recurso que sofreu a maior sobrecarga no sistema.

Sinal de Competição: O comando ollama ps registrou 100% de uso de CPU de forma constante durante a inferência do modelo qwen2.5:1.5b.

Impacto: Como o processador (AMD Ryzen 5 5500) possui 6 núcleos e 12 threads e o Ollama disparou mais de 15 threads simultâneas, os núcleos físicos foram completamente disputados, gerando um gargalo no processamento matemático da IA.

h. Disputa por Sincronização de Memória (*Overhead de Threads*)

Embora houvesse memória RAM suficiente disponível fisicamente (~15 GB no WSL, com o modelo ocupando apenas 1.2 GB em execução), houve uma disputa massiva a nível de software sobre como essa memória era acessada.

Sinal de Competição: A chamada de sistema futex abocanhou 88.29% do tempo total de execução da ferramenta (strace).

Impacto: O comando futex serve para bloquear/sincronizar o acesso à memória RAM compartilhada entre as threads do processo. Esse percentual altíssimo prova que as threads passaram a maior parte do tempo competindo entre si e esperando o desbloqueio da memória, em vez de realizarem cálculos úteis.

i. **Subutilização por Incompatibilidade de GPU e VRAM**

Diferente dos outros componentes, o sinal aqui não foi de disputa por esgotamento, mas sim de uma barreira técnica que causou a total exclusão do recurso.

Sinal de Competição: "Mesmo existindo vram o sistema usou a cpu para processamento". A placa de vídeo AMD RX 7600 de 8GB foi limitada a usar apenas o dispositivo básico de renderização virtualizado do Windows.

Impacto: A VRAM não pôde ser competida ou utilizada devido à incompatibilidade dos drivers da AMD com o ambiente WSL2. Isso sobrecarregou diretamente a CPU e a memória RAM do sistema.

Esse impacto já vimos no texto.

j. **Competição na Rede (Tráfego de Dados)**

A concorrência de rede apareceu com a chegada de requisições paralelas.

Sinal de Competição: O sistema foi configurado para lidar com 3 requisições simultâneas e 3 conversas paralelas, com o serviço escutando ativamente na porta local 11434.

Impacto: A chamada de sistema accept4 consumiu 5.41% (como vemos abaixo no início do strace-resumo) do tempo total gerenciando a entrada dessas conexões concorrentes. Como a CPU estava em 100%, o tempo para processar e responder cada pacote de rede foi estendido.

strace-resumo.txt

```
% time      seconds  usecs/call   calls   errors syscall -----
-----
88.29  74.398560    1733   42924   4505 futex
5.41   4.555932    51190    89     5 accept4
```

k. **Armazenamento (Sem Sinais de Competição)**

O armazenamento foi o único recurso que trabalhou com folga total. O comando `df -h`, conforme figura 22, mostrou que o disco possuía 1007 GB totais e apenas 1% em uso. Como o modelo ocupava meros 941 MB em disco e foi carregado na memória no início da execução, não houve disputa por leitura/escrita (I/O Wait) que impactasse o teste.

Armazenamento (1007G)

```
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/system32$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdf         1007G  5.6G  951G   1% /
biaah@DESKTOP-IDA0AM7:/mnt/c/WINDOWS/system32$ |
```

Figura 22: Armazenamento.

Segue abaixo tabela comparativa da competição de recursos.

Recurso	Status do Recurso	Evidência Científica / Comando	Impacto no Desempenho e Comportamento
CPU	Saturação Crítica	ollama ps (100% CPU de uso constante)	Os 6 núcleos e 12 threads do Ryzen 5 foram totalmente disputados pelas mais de 15 threads do Ollama.
Memória RAM	Disputa por Sincronização	strace: 88.29% do tempo na chamada futex	Embora houvesse espaço (~15GB), as threads perderam tempo esperando umas pelas outras para

			acessar a RAM compartilhada.
GPU / VRAM	Exclusão Técnica	lspci: Microsoft Basic Render Driver Nota da Config 3	A GPU AMD RX 7600 (8GB) foi totalmente ignorada devido à incompatibilidade dos drivers com o WSL2, jogando todo o peso na CPU.
Rede	Competição por Conexão	strace: 5.41% do tempo na chamada accept4	Gargalo no gerenciamento das 3 requisições simultâneas e 3 conversas paralelas que chegavam na porta 11434.
Armazenamento	Folga Total	df -h: 1% de uso do volume de 1007G	Sem sinais de competição. O modelo de 941M foi lido rapidamente e mantido em cache na inicialização.

Tabela 6: Tabela de Análise e Competição de Recursos (Ollama via WSL2)

Vamos ver na tabela abaixo um comparativo entre os dois ambientes, com GPU e sem GPU:

Recurso Analisado	Ambiente Com GPU (NVIDIA RTX 3050)	Ambiente Sem GPU (CPU / WSL / AMD RX 7600)
GPU / VRAM	Competição Saudável (Hardware Dedicado): • Utilização cravada em 97% continuamente. • Alocação fixa na VRAM sobe de	Exclusão Técnica Absoluta (Gargalo por Incompatibilidade): • A GPU física (AMD RX 7600 de 8GB) é totalmente ignorada e

	1,2 GB para 1,4 GB (+200 MB) ao expandir o contexto.	limitada ao driver virtual básico do Windows devido a falhas no WSL2.
CPU / Processador	Folga Operacional: • Sincroniza buffers e gerencia a API de alto nível, deixando o cálculo pesado com os núcleos CUDA.	Saturação Crítica Máxima: • Uso travado em 100% da CPU de forma constante. • As mais de 15 threads do Ollama esgotam totalmente as 12 threads físicas/lógicas do Ryzen 5.
Memória RAM	Competição Residual Controlada: • Ocupação global sobe de 8,5 GiB para 9,2 GiB (+700 MiB) para gerenciar o tráfego dos tokens dos 3 clientes em paralelo.	Gargalo Massivo de Sincronização (Overhead): • Memória física disponível (~15 GB) opera com folga de espaço, mas as threads travam o sistema disputando dados na RAM compartilhada via chamada futex (88.29% do tempo).
Rede	Sem Sinais de Estresse: • Requisições originadas em localhost (loopback local), eliminando qualquer concorrência ou latência física.	Competição por Tratamento Concorrente: • A chamada accept4 consome 5.41% do tempo total capturando as conexões paralelas na porta 11434. Com a CPU em 100%, o tempo para responder a rede se estende.
Armazenamento	Folga Total: • Modelo previamente carregado. Escrita contínua de telemetria leve (kilobytes) no arquivo gpu.csv sem causar gargalo de I/O.	Folga Total: • O volume de 1007 GB registra apenas 1% de uso. O modelo de 941 MB é mantido em cache após a carga inicial, sem gerar gargalos de disco.

Tabela 7: Tabela Comparativa de Competição por Recursos – Ollama.

1.9 Como entradas ou contextos maiores afetaram o desempenho e o uso de recursos?

Com base nos experimentos documentados (especialmente a comparação entre as configurações A/B e a configuração C) para o ambiente com GPU, o aumento do contexto de 4096 para 8192 tokens, conforme discutido em textos anteriores, impactou negativamente o tempo de preparação, mas manteve a eficiência de processamento após a inicialização. Como podemos observar a seguir.

Impacto no Desempenho

Gargalo na Inicialização: O tempo de carregamento inicial disparou para 3,31 segundos (Teste C-1). Na execução padrão com contexto menor, esse tempo era quase instantâneo (0,0006 s).

Velocidade de Geração Preservada: Surpreendentemente, uma vez superada a fase de carga, a velocidade de processamento pura não foi prejudicada. O modelo atingiu sua maior taxa de vazão no contexto longo, registrando 122,15 tokens/s (contra 118,54 tokens/s no contexto padrão), como também já vimos em artefatos anteriores.

Impacto no Uso de Recursos (Hardware)

Aumento da Alocação Estática de VRAM: Para suportar o dobro de contexto, o Ollama expandiu o tamanho do modelo alocado na memória de vídeo da GPU de 1,2 GB para 1,4 GB, conforme já visto. Isso ocorre devido à reserva prévia de espaço para o mecanismo de KV Cache.

Estabilidade da RAM Geral: Diferente da VRAM da GPU, a memória RAM convencional do sistema não sofreu pressão adicional decorrente do tamanho do contexto, permanecendo estável em 9,1 GiB ocupados antes e depois do teste, conforme figura 23 abaixo:

Estado da RAM antes/depois: `memoria-antes.txt`

===== B-1: MEMÓRIA ANTES =====						
	total	used	free	shared	buff/cache	available
Mem:	15Gi	8.5Gi	1.2Gi	396Mi	6.5Gi	7.0Gi
Swap:	4.0Gi	1.4Gi	2.6Gi			

, `memoria-depois.txt`

===== B-1: MEMÓRIA DEPOIS =====						
	total	used	free	shared	buff/cache	available
Mem:	15Gi	9.2Gi	557Mi	602Mi	6.7Gi	6.3Gi
Swap:	4.0Gi	1.4Gi	2.6Gi			

Figura 23: Estabilidade da RAM Geral.

Já para o ambiente sem GPU, o comportamento apresentado de acordo com os testes realizados com contextos maiores (fixados em 4096 tokens) e o aumento da carga de entradas por meio de 3 requisições simultâneas e 3 conversas paralelas afetaram drasticamente o desempenho do sistema das seguintes formas:

- **Degradação do Tempo de Resposta (Latência):** O tempo base de carregamento e resposta do modelo, que em condições ideais e isoladas levava cerca de 10 segundos, aumentou substancialmente com o volume maior de dados e acessos simultâneos. Como o poder de processamento precisou ser dividido para computar os longos blocos de texto (tokens de contexto) de múltiplos usuários ao mesmo tempo, a velocidade de geração de texto caiu.
- **Aumento Exponencial do Overhead de Sincronização (CPU e RAM):** Entradas concorrentes e contextos maiores exigem mais trocas de dados na memória. Como o processamento foi executado inteiramente na CPU, isso gerou uma disputa massiva entre as mais de 15 threads do Ollama. A prova técnica disso é que a chamada de sistema `futex` consumiu 88.29%(conforme vimos anteriormente) de todo o tempo de execução do programa. Ou seja, o processador passou a maior parte do tempo gerenciando e enfileirando o acesso das threads aos contextos maiores na memória RAM compartilhada, em vez de computar os tokens da IA.
- **Saturação Total do Hardware por Falta de Escalonamento em GPU:** Em arquiteturas de IA ideais, contextos maiores são absorvidos pela memória ultra-rápida (VRAM) da placa de vídeo, como não temos aqui, o sistema usou a cpu para processamento, devido à incompatibilidade de drivers, conforme já discutimos. Isso fez com que o processador operasse travado em 100% de uso de CPU de forma ininterrupta, transformando a CPU e a RAM do sistema no gargalo absoluto do ambiente.

A tabela a seguir realiza comparativo entre os dois ambientes, com GPU e sem GPU, para o Impacto de Entradas e Contextos Maiores no Ollama:

Critério Analisado	Ambiente Com GPU (NVIDIA RTX 3050)	Ambiente Sem GPU (CPU / WSL / AMD RX 7600)
Tempo de Inicialização / Carga	Penalidade de Carga Fixo: • Salta de 0,0006s para 3,31s para alocar previamente o espaço estático do contexto.	Arrastado / Degradado: • O tempo base (que já era de ~10s de forma isolada) sobrecarrega e cresce progressivamente.
Vazão após Inicialização	Preservada / Otimizada: • Atinge sua maior eficiência com 122,15 tokens/s (ganho de velocidade sutil em sequências longas).	Penalizada e Dividida: • O cálculo dos grandes blocos satura o chip, reduzindo a velocidade de geração individual por usuário.
Mecânica de Armazenamento	KV Cache Dedicado (VRAM): • O consumo estático na placa gráfica expande de 1,2 GB para 1,4 GB (+200 MB).	Disputa via Threads (RAM Hospedeiro): • Os blocos de dados pesados são empurrados para a memória RAM compartilhada do sistema.
Gargalo a Nível de SO	Mínimo: • A memória RAM do sistema permanece estável e controlada na faixa de 9,1 GiB.	Colapso por Bloqueio (futex): • A sincronização do contexto longo e das requisições exige 88.29% do tempo da CPU em travas mútuas.
Saturação do Processador	Descarregado: • O chip principal não sofre pressão extra, delegando as matrizes longas aos núcleos de computação gráfica.	Saturação Crítica Contínua: • Processador AMD Ryzen fica travado em 100% de uso, limitando a capacidade física de escalonamento.

Tabela 8: Impacto de Entradas e Contextos Maiores no Ollama.

1.10 Como a arquitetura local contribui para privacidade, disponibilidade, autonomia e controle de dados? Quais riscos permanecem?

A execução de Modelos de Linguagem de Grande Porte (LLMs) em uma arquitetura local — como demonstrado no relatório através do uso nativo do Ollama e da GPU RTX 3050 em ambiente Linux — transforma radicalmente a governança sobre a Inteligência Artificial. Abaixo está a análise de como essa infraestrutura local, no ambiente com GPU, contribui para a operação e quais riscos ainda persistem:

Benefícios (Privacidade e Controle)

Privacidade Total: Os dados do prompt e da resposta ficam restritos à máquina local. Como o ambiente roda nativo no Linux, nenhuma informação é enviada para servidores externos.

Disponibilidade e Autonomia: O sistema funciona 100% offline, sem depender de internet. O controle é total, permitindo alterar o contexto de 4096 para 8192 tokens de forma livre, conforme visto anteriormente.

Riscos (Hardware e Desempenho)

Gargalo sob Alta Demanda: Os recursos físicos são limitados. Colocar apenas 3 requisições simultâneas fez o tempo de resposta disparar de 3,56 segundos para 16,54 segundos.

Limite de Memória (VRAM): Expandir o contexto exigiu mais memória da GPU (subindo para 1,4 GB). Se o limite da placa (6144 MiB) for ultrapassado, o sistema sofre um crash.

Desgaste e Consumo: A execução intensa eleva os custos físicos, fazendo a GPU atingir picos de 69,08 W de consumo e 59 °C de temperatura, como vimos nos relatórios dos testes, das questões acima.

Para o ambiente sem GPU temos:

Infraestrutura e Limitação da GPU

O teste foi feito em um processador AMD Ryzen 5 5500 com 15 GB de RAM no WSL2 (Linux dentro do Windows). O grande gargalo técnico foi a incompatibilidade dos drivers da GPU AMD RX 7600 (8GB) com o WSL2. Isso impediu o uso da memória de vídeo (VRAM), forçando o sistema a rodar o modelo de IA 100% na CPU.

Impacto do Modelo no Hardware

O modelo utilizado foi o qwen2.5:1.5b. Por ser um modelo leve e compactado (quantizado), ele ocupou apenas 941 MB em disco e consumiu 1.2 GB de memória ao ser carregado. Seu tempo de inicialização foi de 1 segundo e o carregamento levou cerca de 10 segundos, conforme já visto anteriormente.

Processos e o Custo da Concorrência

Ao rodar, o Ollama criou um processo principal (PID 1294) que disparou mais de 15 threads para lidar com as tarefas, conforme vimos nas questões 4 e 5 deste documento. Quando o sistema foi testado com 3 requisições simultâneas e contextos de 4096 tokens, o desempenho caiu. Como não havia GPU, as threads começaram a disputar intensamente a CPU e a memória RAM. A prova disso está nas chamadas de sistema monitoradas (strace): a chamada futex consumiu 88.29% de todo o tempo de execução, já descrito no resumo strace-resumo.txt,

em questões anteriores. Isso significa que o processador passou quase todo o tempo organizando e enfileirando as threads (esperando umas pelas outras) em vez de focar no cálculo da IA.

Vantagens e Riscos da Execução Local

Vantagens: Garante privacidade total (os dados não saem da máquina), disponibilidade (funciona sem internet na porta local 11434) e total autonomia sobre o modelo.

Riscos: O sistema está vulnerável a travamentos por lentidão extrema sob alta carga (gargalo de CPU) e depende totalmente da energia e da segurança física daquela máquina local.

Em um comparativo entre os modelos com e sem GPU, temos a seguinte tabela:

Vetor de Análise	Ambiente Com GPU (NVIDIA RTX 3050)	Ambiente Sem GPU (CPU / WSL / AMD RX 7600)
Privacidade e Governança	Absoluta e Nativa: Os dados permanecem restritos ao hardware local. Sem tráfego externo ou risco de vazamento para nuvens de terceiros.	Absoluta e Isolada: Isolamento total de dados no localhost (porta 11434), garantindo compliance regulatório estrito.
Disponibilidade e Conectividade	100% Autônoma (Offline): Independência total de links de internet ou oscilações de APIs comerciais.	100% Autônoma (Offline): Funcionamento garantido sem internet, mas altamente suscetível a travamentos por esgotamento de hardware.
Controle de Parâmetros	Flexibilidade Total: Liberdade para estender o contexto (ex: 4096 para 8192 tokens) gerenciando diretamente o impacto na VRAM.	Controle Limitado por Hardware: Liberdade conceitual de configuração, mas limitada pela barreira de desempenho crítico da CPU.

Riscos de Disponibilidade Física	Degradação por Fila (Latência): Sob alta demanda (3 requisições), a responsividade individual cai (até 16,54s), mas o serviço não colapsa.	Vulnerabilidade por Travamento Extremo: Alta concorrência gera saturação em 100% da CPU, com risco real de congelamento do sistema hospedeiro.
Riscos de Sustentabilidade Física	Estresse Térmico e Energético: Exige dissipação ativa do hardware gráfico (picos de 69,08 W e temperatura máxima de 59 °C).	Degradação Prematura do Host: Exige esforço contínuo do processador principal e alto custo de sincronização de software (futex a 88.29%).
Dependência Crítica de Infraestrutura	Limite Físico de Silício (VRAM): Risco de crash imediato do sistema caso o modelo/contexto ultrapasse o teto físico de memória da placa (6144 MiB).	Barreira de Compatibilidade de Software: Totalmente dependente da segurança física e energia do host, agravado pela exclusão da placa de vídeo AMD.

Tabela 9: Comparativa: Governança, Autonomia e Riscos Locais.

1.11 Como o sistema poderia se comportar diante de múltiplos usuários ou requisições simultâneas?

Diante de múltiplos usuários ou requisições simultâneas, o comportamento do sistema, com GPU, pode ser resumido em três pontos principais:

Fila no Software (Aumento de Latência): Embora a velocidade de geração por usuário se mantenha estável em cerca de 119 tokens/s, o tempo total de espera disparou de 3,56s para até 16,54 segundos, conforme analisado. Isso ocorre porque os usuários entram em uma fila assíncrona, e o tempo de carregamento inicial salta de quase zero para 1,96 segundos para gerenciar o contexto de cada um.

Saturação da GPU: A placa de vídeo (RTX 3050) é empurrada imediatamente para o seu teto de processamento, operando em 97% de utilização contínua. O consumo

elétrico sobe para o pico de 69,08 W e a temperatura sobe cumulativamente, atingindo 59 °C, como analisamos nos dados gerais de questões passadas.

Pressão na Memória RAM: Cada requisição simultânea exige buffers extras do sistema operacional. No teste com 3 usuários, o consumo de RAM geral aumentou em 700 MiB (passando de 8,5 GiB para 9,2 GiB usados, conforme já visto), indicando que uma escala maior de usuários exigirá muita memória do sistema.

Já para o ambiente sem GPU, temos que:

Lentidão Extrema (Efeito Fila): Como o processador já opera em 100% de uso de CPU com poucos acessos, adicionar mais usuários dividirá a capacidade matemática do chip. O tempo de resposta individual aumentaria de forma drástica e exponencial.

Paralisia por Disputa de Threads: O processador começaria a gastar quase todo o seu tempo apenas tentando organizar a fila de execução das mais de 15 threads do Ollama, em vez de processar a IA. Isso é previsto pela chamada futex (88.29% do tempo, que vimos no resumo de questões anteriores), que indica threads travadas esperando umas pelas outras para acessar a memória RAM.

Risco de Queda Geral (Esgotamento de RAM): O modelo consome 1.2 GB base, mas cada nova conversa consome memória extra para guardar o histórico de contexto (de até 4096 tokens), conforma já analisamos. Com muitos usuários simultâneos, o limite de ~15 GB de RAM do WSL2 poderia estourar, fazendo o sistema operacional derrubar o Ollama repentinamente para evitar o travamento do computador.

Em uma comparação entre os dois ambientes temos:

Vetor de Análise	Ambiente Com GPU (NVIDIA RTX 3050)	Ambiente Sem GPU (CPU / WSL / AMD RX 7600)
Mecanismo de Escalonamento	Fila Assíncrona via Software: Os usuários entram em uma fila gerenciada pelo Ollama. A velocidade pura de geração é mantida (~119 tokens/s), mas a latência individual é penalizada por ordem de chegada.	Divisão Direta de Recursos: O poder computacional da CPU é fatiado entre os usuários. A capacidade matemática do chip é reduzida proporcionalmente, gerando lentidão extrema para todos ao mesmo tempo.

Latência e Tempo de Espera	Aumento Controlado / Escalonado: O tempo total de resposta sobe do padrão de 3,56s para patamares de 5,80s, 12,75s e até 16,54s, dependendo da posição do cliente na fila.	Crescimento Exponencial Rápido: O tempo de resposta individual explode rapidamente de forma imprevisível à medida que novos usuários entram.
Comportamento do Processador	Teto Operacional Estabilizado: A GPU é empurrada para 97% de utilização contínua, absorvendo o estresse da concorrência e isolando a CPU do host.	Paralisia por Disputa de Threads: CPU trava em 100% de uso. As mais de 15 threads passam a gastar mais tempo se coordenando via futex (88.29% do tempo) do que processando IA.
Impacto e Estresse na Memória	Pressão Controlada por Buffers: Cada contexto simultâneo exige espaço extra de tráfego, gerando um acréscimo linear mensurável na RAM global (+700 MiB, saltando de 8,5 GiB para 9,2 GiB).	Risco de Queda Geral (OOM Crash): Cada nova conversa exige memória extra para armazenar o histórico de contexto longo. Sob alta demanda, há risco real de estourar o limite de ~15 GB do WSL, ativando o Out of Memory Killer.
Tempo de Carregamento Inicial	Salta do tempo quase instantâneo (0,0006s) para a faixa estável de 1,96 segundos em todas as requisições paralelas para preparar os buffers.	Degradado e arrastado severamente, uma vez que a CPU precisa realizar I/O e computação matemática concorrentemente sem aceleração.
Métricas Físicas de Infraestrutura	• Consumo atinge o teto de 69,08 W. • Temperatura sobe cumulativamente a 59 °C.	• Sobrecarga térmica e elétrica contínua no chip do host. • Risco de congelamento completo do sistema operacional.

Tabela 10: Comparativa dos ambientes com e sem GPU: Comportamento sob Múltiplos Usuários Simultâneos.

1.12 Quais limitações do hardware, do runtime, da camada de aplicação e do modelo afetam a validade dos resultados?

Para analisar a validade científica e estatística dos resultados apresentados nos relatórios, é necessário identificar as limitações estruturais e metodológicas presentes em cada camada do ecossistema de testes.

Abaixo estão as limitações divididas por categoria, para o ambiente com GPU, que restringem o escopo e a generalização dos dados coletados:

Limitações do Hardware (GPU e Sistema)

Amostragem Reduzida de Logs (Granularidade): O arquivo de monitoramento `gpu.csv`, como vimos na questão 6, registra métricas em intervalos fixos de aproximadamente 200 milissegundos (0,2 segundos). Fenômenos rápidos, como micro-picos de consumo de energia (Watts) ou variações instantâneas de tensão, não são capturados devido à baixa taxa de amostragem.

Gargalo Térmico Oculto (Thermal Throttling): A temperatura máxima registrada subiu para 59 °C, conforme discutimos na questão 7. Embora esteja dentro da margem segura para uma RTX 3050, o relatório não monitora a curva de clock da GPU. Não é possível validar se a placa reduziu suas frequências de operação para manter a temperatura controlada, o que afetaria a velocidade de tokens/s.

Falta de Isolamento de Background: Processos gráficos pesados como o Xorg (282 MiB), `gnome-shell` (68 MiB) , vistos na questão 3, e instâncias do `steamwebhelper` e `steam` estavam rodando na GPU em paralelo aos testes. Essa atividade concorrente do sistema operacional introduz ruído e variações imprevisíveis nos resultados de consumo de VRAM e processamento.

Limitações do Runtime (Ollama e Drivers)

Mecanismo Opaco de Agendamento: O Ollama gerencia requisições concorrentes de forma automática em seu backend. No Teste B-1, conforme já vimos, o salto fixo de 1,96 s no tempo de carregamento para as três requisições indica um atraso artificial de fila por software, mascarando o tempo real que o hardware levaria para responder se utilizasse uma biblioteca de concorrência pura (como `vLLM` ou `TGI`).

Ausência de Dados do KV Cache por Requisição: O runtime não discrimina quanta VRAM foi gasta especificamente para o Key-Value Cache de cada usuário na concorrência, fornecendo apenas uma métrica global do processo (`llama-server`).

Limitações da Camada de Aplicação e Metodologia

Número Insuficiente de Repetições: Embora a Configuração A mencione três repetições (A-1, A-2 e A-3), as Configurações B e C apresentam dados de apenas uma única execução (Teste B-1 e Teste C-1),

conforme vimos nas questões anteriores. Sem múltiplas repetições para todos os cenários, é impossível calcular desvios-padrão ou margens de erro, o que reduz a validade estatística da comparação.

Estimulação Homogênea (Prompts Idênticos): No teste de concorrência, todas as três requisições disparadas simultaneamente possuíam exatamente 66 tokens de prompt, conforme os testes apresentados na questão 6. Na vida real, usuários concorrentes enviam tamanhos de texto diferentes. prompts idênticos geram um comportamento otimizado no cache de leitura do sistema, distorcendo a simulação de uso real.

Limitações do Modelo (qwen2.5:1.5b-instruct)

Subutilização de Recursos por Tamanho Reduzido: Por possuir apenas 1.5 bilhão de parâmetros, o modelo é extremamente leve e exige apenas 1,2 GB a 1,4 GB de VRAM, operando folgado na placa de 6 GB, conforma já vimos. Os resultados de estabilidade térmica e consumo elétrico válidos para este modelo não se aplicam a modelos maiores (como os de 7B ou 8B), que saturariam a placa de vídeo e apresentariam comportamentos de gargalo totalmente diferentes.

Para o ambiente sem GPU, também vamos resumir em 4 pontos, como segue:

Do Hardware: O teste rodou 100% na CPU, ignorando a placa de vídeo AMD RX 7600 (8GB) por incompatibilidade de drivers com o WSL2. Isso invalida o teste como reflexo de um cenário real de mercado, onde o uso de GPU é o padrão. Além disso, rodar no WSL2 divide recursos com o Windows, gerando oscilações nos tempos medidos.

Do Runtime (Ollama): O software disparou mais de 15 threads que travaram o processador em disputas internas de sincronização, gerando o gargalo na chamada futex (88.29% do tempo), conforme já analisado. Esse desperdício de processamento mascara a real eficiência do framework.

Da Camada de Aplicação: Os testes usaram uma carga muito baixa (3 requisições simultâneas) para avaliar o estresse real do sistema. Além disso, o relatório mediu apenas dados do sistema operacional (como uso de CPU), mas não trouxe métricas essenciais de IA, como a velocidade de geração de texto (Tokens por Segundo).

Do Modelo (qwen2.5:1.5b): Por ser um modelo minúsculo (consome apenas 1.2 GB de RAM), o comportamento de armazenamento e memória medido nele não serve para prever como o sistema se comportaria com modelos comerciais padrões (que são de 5 a 10 vezes maiores).

Em uma comparação entre os dois temos que:

Camada do Ecossistema	Ambiente Com GPU (NVIDIA RTX 3050)	Ambiente Sem GPU (CPU / WSL / AMD RX 7600)
Limitações do Hardware	• Granularidade dos logs: O gpu.csv grava a cada ~200ms, perdendo micro-picos de energia (W). • Thermal Throttling oculto: Sem monitorar a curva de clock da GPU, não se sabe se houve perda induzida de desempenho sob 59 °C. • Falta de Isolamento: Processos gráficos concorrentes (Xorg, gnome-shell e Steam) introduzem ruído.	• Desvio de Escopo Técnico: Execução travada 100% na CPU por incompatibilidade de drivers, invalidando o uso como referência padrão de mercado. • Falta de Isolamento Host-Guest: O subsistema WSL2 divide recursos físicos diretamente com o Windows hospedeiro, gerando oscilações.
Limitações do Runtime	• Mascaramento de Fila: O delay fixo de 1,96s na carga oculta o tempo real que uma biblioteca com concorrência nativa e pura entregaria.	• Overhead de Sincronização: O framework disparou mais de 15 threads concorrentes que paralisaram o chip Ryzen

Limitações da Aplicação e Metodologia	• Métricas Opacas: O Ollama reporta apenas o consumo global do llama-server, sem isolar o gasto de VRAM do KV Cache por usuário.	em esperas de kernel (futex absorveu 88.29% do tempo).
	• Baixa Amostragem Estatística: Apenas a Configuração A possui três repetições. As configurações B e C possuem execução única, impossibilitando o cálculo de desvios-padrão. • Viés de Estímulo: Uso de prompts idênticos (66 tokens), gerando cenário otimizado de cache.	• Carga Subdimensionada: O estresse foi avaliado com uma quantidade de requisições extremamente baixa (3 simultâneas). • Ausência de Métricas de IA: O teste mediu parâmetros clássicos do sistema operacional, mas omitiu dados de vazão de IA (Tokens por Segundo).
Limitações do Modelo Utilizado	• Subdimensionamento de Carga: O modelo qwen2.5:1.5b é extremamente leve e ocupa pouca VRAM (1,2 GB a 1,4 GB). Os resultados não se aplicam a modelos comerciais maiores (7B ou 8B).	• Inabilidade de Projeção: O modelo testado é pequeno (ocupa 1.2 GB de RAM e 941 MB em disco). É incapaz de prever o comportamento de modelos comerciais padrão (5 a 10 vezes maiores).

Tabela 11: Relatório de Consistência Técnica e Validade Estatística.

Como principais conclusões sobre a validade, podemos descrever:

Ambiente Com GPU: Viés de Otimização e Baixa Repetibilidade

O experimento baseado em hardware NVIDIA apresenta uma prova de conceito funcional, mas com restrições de rigor metodológico. O uso de prompts idênticos de 66 tokens ativa mecanismos de reaproveitamento de cache que mascaram a latência real de um cenário com múltiplos usuários enviando textos distintos. Adicionalmente, a falta de repetições sequenciais com cálculo de desvio-padrão nas configurações de concorrência impede que as oscilações provocadas pelos processos de background sejam tratadas estatisticamente, limitando os dados a um retrato pontual da execução.

Ambiente Sem GPU: Desvio Estrutural de Cenário

No ecossistema CPU via WSL2, as limitações alteram a natureza do teste. A total exclusão da placa AMD RX 7600 por barreiras de driver transforma o experimento em um ensaio de tolerância a falhas e comportamento anômalo, e não em uma medição padrão de arquitetura de IA local. Como o relatório monitorou apenas métricas de sistema operacional e omitiu a velocidade de geração de tokens por segundo, não há insumos técnicos para validar a qualidade da experiência do usuário final, restando apenas a comprovação do colapso de concorrência gerado pelo overhead da chamada futex.

1.13 Quais informações geradas por IA precisaram ser verificadas, corrigidas ou rejeitadas?

Com base nas análises, para o ambiente com GPU, a resposta da IA foi conceitualmente correta e bem-sucedida, não havendo alucinações textuais que precisassem ser corrigidas ou rejeitadas.

O que exigiu verificação e esclarecimento analítico foram duas métricas geradas pelas APIs de medição:

O "Tamanho" do Modelo no ollama ps: As tabelas indicam que o tamanho do modelo mudou de 1.2 GB para 1.4 GB. Essa informação precisou ser checada com o nvidia-smi, para entender que o arquivo do modelo continuava igual, e que a diferença era apenas a reserva de memória extra (KV Cache) para o contexto maior, conforme vimos quando analisamos o estado antes e depois das execuções.

O "Tempo de Carregamento" na Concorrência: A API registrou que cada uma das 3 requisições simultâneas levou 1,96 segundos para carregar, vimos quando analisamos as questões sobre os testes. Esse dado precisou ser contextualizado pelo analista: como o modelo já estava na VRAM, esse tempo não foi de leitura de disco, mas sim o atraso da fila do software tentando organizar os pedidos recebidos ao mesmo tempo.

Já para o ambiente sem GPU, vamos resumir em 3 pontos, são eles:

Descartado (O uso da GPU): O sistema precisou rejeitar qualquer tentativa de aceleração gráfica por hardware. Como os drivers da AMD RX 7600 (8GB) são incompatíveis com o WSL2, o comando lspci identificou apenas um driver genérico da Microsoft, forçando o descarte da GPU e a migração de todo o trabalho para a CPU, como já demonstrado anteriormente.

Validado (O tamanho real do modelo): Foi necessário verificar a diferença de tamanho do modelo entre o disco e a memória. Enquanto os arquivos na pasta do Ollama ocupavam 941 MB em disco, o comando ollama ps validou que, ao ser carregado para execução na memória RAM, o modelo se expandia para 1.2 GB, conforme já vimos.

Corrigido (A causa da lentidão): A análise do strace corrigiu a suposição de que a lentidão com 3 requisições simultâneas vinha do atraso de rede ou de leitura do disco. O relatório provou que o real culpado era interno: 88.29% do tempo foi gasto na chamada futex, corrigindo o diagnóstico para um problema de disputa de threads na CPU, conforme analisamos o evidencia abaixo no “strace-resumo.txt” na questão 7.

Em uma comparação entre os dois ambientes temos a seguinte tabela:

Categoria da Informação	Ambiente Com GPU (NVIDIA RTX 3050)	Ambiente Sem GPU (CPU / WSL / AMD RX 7600)
Informações Verificadas	Mecânica do KV Cache (ollama ps): Foi verificado o motivo pelo qual o modelo mudou de 1.2 GB para 1.4 GB. O cruzamento com o nvidia-smi provou que o peso do arquivo físico era idêntico, tratando-se apenas da expansão da reserva estática de contexto.	Expansão em Memória (ollama ps): Foi validada a discrepância volumétrica onde o modelo ocupa 941 MB em disco, mas se expande para 1.2 GB de alocação ativa quando carregado na memória RAM convencional do hospedeiro.

Informações Corrigidas	Natureza do Tempo de Carga: • Foi retificado o entendimento sobre o tempo de carregamento de 1,96s sob concorrência. O modelo já estava na VRAM; logo, o dado foi corrigido de 'leitura de disco' para 'overhead/atraso artificial da fila de software'.	Diagnóstico do Gargalo Sistêmico: • Foi corrigida a suposição inicial de que a lentidão sob concorrência vinha de I/O de disco ou rede. O strace provou que o vilão era interno: 88.29% do tempo foi gasto em travas futex de threads.
Informações Rejeitadas / Descartadas	Inexistência de Alucinações: • A camada de geração de texto da IA mostrou-se conceitualmente correta e bem-sucedida, não havendo outputs textuais espúrios ou errados para rejeição.	Expectativa de Aceleração Gráfica: • Foi rejeitada qualquer tentativa ou suposição de uso da GPU física. O lspci expôs o Microsoft Basic Render Driver, forçando o descarte da placa AMD devido a incompatibilidades crônicas com o WSL2.

Tabela 12: Relatório de Auditoria Analítica e Validação de Dados.

Conclusão

Vimos que o Ambiente com GPU (NVIDIA RTX 3050), demonstrou alta eficiência. O processamento pesado foi absorvido pela placa gráfica, mantendo a velocidade estável em ~119 tokens/s. O impacto do aumento de usuários ou contexto gerou apenas uma fila no software e aumento previsível da VRAM (KV Cache).

Já o Ambiente sem GPU (CPU / WSL2 / AMD RX 7600) sofreu um colapso de desempenho devido à incompatibilidade de drivers, ignorando a placa de vídeo. A CPU ficou travada em 100% de uso, e o sistema gastou 88,29% do tempo apenas organizando as threads (chamada futex) em vez de processar a IA, gerando lentidão extrema e risco de travamento.

A Governança Local com a arquitetura local cumpriu com sucesso o papel de garantir privacidade total, funcionamento offline e controle absoluto dos dados.

E quanto as Limitações Metodológicas temos um viés de otimização devido ao uso de prompts idênticos. Por testar um modelo muito leve (1.5B), os resultados não podem ser projetados para modelos comerciais maiores (7B ou 8B), que saturariam o hardware rapidamente.

Assim entendemos que as análises foram bem sucedidas e bastante relevantes ao contexto do trabalho pois analisamos dois ambientes distintos e é possível perceber que a execução local é vantajosa para a segurança dos dados, mas

depende obrigatoriamente de uma GPU com drivers compatíveis. Rodar modelos puramente por CPU/RAM é inviável para cenários reais de uso concorrente.

2. Referências

1. HUGGING FACE Community, Qwen2.5-1.5B-Instruct, portal da comunidade Hugging Face. Disponível em: <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>. Acessado em: 11 set. 2026.
2. FRANTAR Elias. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers, portal Arxiv, 22 mar. 2023. Disponível em: <https://arxiv.org/abs/2210.17323>. 11 set. 2026.
3. HUI, Binyuan, Qwen2.5 Technical Report, portal Arxiv, 3 jan. 2025. Disponível em: <https://arxiv.org/abs/2412.15115>, Acessado em: 11 set. 2026.
4. OLLAMA, qwen2.5:1.5b-instruct-q4_K_M portal ollama. Disponível em: https://ollama.com/library/qwen2.5:1.5b-instruct-q4_K_M. Acessado em: 11 set. 2026.

3. Declaração de Uso de Inteligência Artificial

Cada integrante do grupo declara, individualmente, o uso (ou não uso) de ferramentas de Inteligência Artificial (IA) na elaboração desta atividade, conforme descrito a seguir:

Evelyn Hilthegarden Santana de Jesus

- **Ferramenta utilizada:** Claude (Anthropic).
- **Versão:** Claude Sonnet 5 (versão gratuita do Claude.ai).
- **Como e onde foi usada:** Utilizada no computador pessoal, como apoio na formatação deste documento (capa, sumário, numeração de páginas e organização geral dos tópicos) e na elaboração desta declaração de uso de IA.

Samyr Teles Sales Silva

Não utilizei nenhuma ferramenta de Inteligência Artificial na elaboração desta atividade.

Matheus de Oliveira Lima

- **Ferramenta utilizada:** Gemini (Google).
- **Versão:** Gemini 3.6 Flash (versão gratuita do Google Gemini).
- **Como e onde foi usada:** Utilizada para auxiliar nas análises e na construção das respostas de cada pergunta.

Rafael de Jesus Santos

Não utilizei nenhuma ferramenta de Inteligência Artificial na elaboração desta atividade.

Luan Feitosa Lima Sátiro

- **Ferramenta utilizada:** Claude (Anthropic).
- **Versão:** Claude Sonnet 5 (versão gratuita do Claude.ai).
- **Como e onde foi usada:** Utilizada para auxiliar no código utilizado na criação dos testes automáticos, dos quais foi responsável.

Beathriz Laurent Carlos Muniz

- **Ferramenta utilizada:** Gemini (Google).
- **Versão:** Gemini 3.6 Flash (versão gratuita do Google Gemini).
- **Como e onde foi usada:** Utilizada para identificar os comandos utilizados no monitoramento dos testes, dos quais foi responsável pela execução.